Scale — Roadmap & Estimation (handoff archive)

Historical archive. This is the completed sequence and estimation record. The current roadmap is `docs/roadmap.typ`; current state is `docs/now.typ`; the remaining order is `docs/backlog-graph.typ`. Do not read this file end to end. The counts below are the counts at the time of the handoff, not the current counts.

Prepared for a successor model. Read `docs/commitments-ledger.typ` (status of every design commitment), `docs/technical-debt-register.typ` (closed; host-type debt), and `docs/decision-records.typ` (E1, D1–D171) before changing semantics.

Read budget. The current state is `docs/now.typ`. This file is the historical sequence and estimation record; read it by section (`doc_get_structure` + `doc_get_section`), never end to end.

0. Verifying current state

```
dotnet build
dotnet run       # the product showcase
dotnet run --project projects/scale-tests/scale-tests.fsproj -- --tests
dotnet run --project projects/scale-tests/scale-tests.fsproj -- --granular-tests
```

At handoff: **build clean (0 warnings/errors) and no LSP diagnostics.** The example suite is **1455 checks** — 1413 example tests and 42 Hedgehog properties, each counted individually. The granular suite adds 33 tests, including eleven Verify snapshots (D23, D24, D148) and nine JIT-only writer checks (D128, D134); `just check` runs both. Discovery is 91 tests. The refinement loop runs in `components/demo/demo/`, executing the emitter's compiled output and discharging the obligations with Z3. See D5–D19. The 56 → 55 drop at W0 was the duplicate-serializer test retiring with its subject; recorded here so the counts reconcile. The move from 197 to 212 is D13's 15 decision-table assertions; no existing check changed. The move from 212 to 245 is W3's 32 sort example checks plus one structural-set-algebra property. The move from 245 to 271 is W4's 24 unification example checks plus two properties (variable-chain resolution and occurs-check). The move from 271 to 275 is W2.1's Scale Basic English checks: thirty-seven parser examples plus one generated sentence property, net three examples after the old W2 corpus is replaced. The move from 275 to 298 is W10's 22 dispatch example checks plus one dispatch property. The move from 298 to 313 is W5's eleven scheduler checks plus W6's two channel checks plus D21's two word metaobject checks. The move from 425 to 441 is W7's eleven event checks plus five error-rendering checks. The move from 441 to 471 is D27's thirty regression checks for the review findings. The move from 552 to 562 is D36's ten sort-constructor checks. The move from 564 to 576 is D41's twelve guard-lattice checks. The move from 576 to 612 is D44's thirty-six cell-system encoding checks. The move from 612 to 623 is D45's eleven closure and lexical-scope checks. The move from 623 to 636 is D46's thirteen class, slot, and instance checks. The move from 636 to 647 is D47's eleven precedence and class-dispatch checks. The move from 1095 to 1115 is D91′s twenty Real and Natural sort checks, and the move from 1115 to 1118 is D92′s three sort-faithful emission checks. The move from 1251 to 1265 is D119′s fourteen coverage and differential checks for total expression lowering, the move from 1265 to 1267 is D120′s two history-scope checks, the move from 1267 to 1268 is D121′s net let gain, the move from 1268 to 1272 is D122′s four lexical-resolution, shadowing, and refusal checks, the move from 1272 to 1276 is D123′s four lambda checks, the move from 1276 to 1278 is D124′s differential-corpus checks, and the move from 1278 to 1288 is D125′s net ten pattern checks (eleven new checks replace the one pattern-match refusal). The granular suite moved from 17 to 26 across D126′s four writer checks, D128′s six harness checks (which replace D126′s four), and D134′s three unit checks. The move from 1288 to 1290 is D133′s two capture checks, the move from 1290 to 1298 is D135′s eight ratchet checks, the move from 1298 to 1300 is D136′s two ABI version checks, the move from 1300 to 1302 is D141′s two content-root checks, the move from 1302 to 1314 is D142′s twelve prose parse and equivalence

checks, the move from 1314 to 1320 is D143′s six comma-form checks, the move from 1320 to 1327 is D144′s seven name, text, and nested-match checks, and the move from 1327 to 1332 is D145′s five sequence and `of`-phrase checks, the move from 1332 to 1335 is D146′s three stack-seam checks, the move from 1335 to 1337 is D148′s two properties, and the move from 1337 to 1338 is D149′s prose CLI check; the granular suite moves from 26 to 27 with the lowered-IR snapshot.

1. Operating discipline (non-negotiable)

The operating discipline now lives in `AGENTS.md`, which is the single home for agent instructions. During alpha, do not run MSBuild, Fantomas, or a NativeAOT build: use SageFs for compile, run, and test. The release gate (`just check`, `just fmt`, `just aot-test`, `just release`) applies only when the owner says a major, public version is being released.

2. Open questions for the owner (answer before/while working)

#	Question	Why it matters
Q1	Is the surface vocabulary **child-facing** by intent?	**Answered: yes, for the sentence layer.** The child-facing product is the input natural language and the emitted F#; the stack words (`nor`, `admit`, `unify`) are the lab surface beneath it. Work on "child-friendliness" targets the sentence layer, not the stack.
Q2	**Hedgehog or FsCheck** for generated property tests?	**Answered: Hedgehog.** Hedgehog 1.0.0 is pinned, `properties.fs` runs 34 properties, and it works in SageFs (proved: `Properties.runAll` passes in 0.7s warm). FsCheck is not adopted. NB: SageFs's live-testing layer discovers tests by attributes (granular/xUnit/NUnit/MSTest/TUnit). `development/scale/granular-tests.fs` wraps the properties (and a representative example slice) as individual granular cases, so `list_tests`/`run_tests` see them; the example aggregate remains the acceptance suite, and `just check` runs both it and the granular tree (the Verify snapshot joins the latter, D23). `dotnet test` still discovers nothing (no VSTest adapter), so use `projects/scale-tests` with `--tests` and `--granular-tests` (D176).
Q3	Sorts surface: `model <sort> <name>` vs a separate `sort` word?	**Answered (D12): a separate word.** A sort is `(Name, Extension)`; `model <name>` collapses it to the name and discards the structural half, which W10 needs. Surface: `[ … ]` structural sets, `:Name` nominal seals. See `docs/set-theoretic-typing.typ`.
Q4	Is "verified compiler" Lean-as-backstop or Lean-as-centerpiece?	**Superseded by D10.** The backstop is Z3 (SMT), not a proof assistant; "verified" stays bounded to "properties are discharged over the theory, the evaluator is sampled".
Q5	~~Which verifier replaces or supplements Lean?~~	**Answered (D10): Z3 in-process via `Microsoft.Z3`; Lean retired as the backstop, retained only as a possible optional export.**
Q6	Is Fable part of the product (a second compile target for the browser) or out of scope?	**Out of scope for now (owner).** D10 records that Fable is **not a verifier**; it earns its place only if the child-facing/browser deployment is a goal (optionally with z3-wasm). It does not block W13.

3. Work items

Effort: 1 session ≈ one focused half-day of agent work including tests and the decision record. Risk: chance the item forces rework later.

ID	Item	Effort	Depends on	Risk	Acceptance criteria
W0	**Safety net + README**	S (<1)	—	none	`git init` + commit; README states thesis, level model, doc map; delete or merge duplicate `serialisation.fs`; remove `failwith` remnants in `Lace`/`IndexTransformation` (convert to `Result` or delete)
W1	**Emitter produces compiling property tests**	S–M (1–2)	Q2	low	"Every player that is alive can jump." emits F# that compiles and passes; type declarations generated from vocabulary `TypeName`/`FieldName`; decision record D5 (framework)
W2 ✓	**Parser generalization**	S–M (1–2)	W1 useful	medium	`recognizePattern` no longer hardcodes `canJump`; quantifier/negation/order respected; `Pattern` reaches the emitter; sentence corpus tests
W2.1 ✓	**Scale Basic English grammar** (D18, D24)	S–M (1)	W2	medium	Explicit sentence grammar with named refusals; `every` is `Capability`, `no` is `Invariant`; numeric `has` is executable end-to-end (D24); `some` is a `Possibility` (D33); articles, pronouns, `when`, negation, `or`, and `all` refused by name; generated-sentence property. See `docs/prose-grammar.typ`.
W3 ✓	**Sorts / value typing**	M (2–3)	Q3 ✓ (D12)	high	image-resident sort registry + tagging relation; a sort is `(option Name, Extension)` (D12); surface `[ … ]` sets and `:Name` seals (`S :Name seal`); `model` is sort-aware; `complement` defaults the universe to a declared sort while the ternary `nor` is unchanged; `ValueSetMismatch`/`ValueSetNotClosed` diagnostics; determinism rule enforced. Landed as D14. See `docs/set-theoretic-typing.typ`.

ID	Item	Effort	Depends on	Risk	Acceptance criteria
W4 ✓	**Unification pillar upgrade**	M (2–3)	W3 ✓	medium	occurs-check; bidirectional flow; substitution resolution over typed terms; structural unify tests. Landed as D15: first-order terms encoded in the image (`-2` compound, `-3` typed), `walk`/`occurs`/substitution, canonical aliases, transactional failure, `resolve`/`term`/`typed`, nominal sort equality.
W5 ✓	**Scheduler replacement**	M (2)	—	medium	Landed: per-process scheduler state threaded through `EvaluationState`, no module-global mutable; `spawn`, `yield`, `halt`, `actors`, `resume`, `kill`; budget exhaustion is a yield; the preemption test really interleaves a parent and child. `scheduler.fs` was rewritten from the sketch — see `docs/evaluator-slicing-and-scheduler.typ`.
W6 ✓	**Channels**	M (2)	W5	medium	channel identities as image entries; send/receive as events; blocking is a distinct outcome, never `Error`
D21 ✓	**Word Metaobjects**	M (1–2)	D16, W6	medium	`WordTag = -6.0` allocated; word metaobjects with token/form/category/active/implRef; dispatcher order: Built-ins ⟶ Word metaobjects ⟶ Generic functions ⟶ User words ⟶ Vocabulary; new word family needs no F# case
W7 ✓	**ChannelEvents**	L (3–4)	W6	high	Landed as D26: image-resident event tensors (`EventTag = -8.0`); `send-event`/`receive-event`/`choose`/`wrap`/`guard`/`with-fallback`/`sync`; a blocked `sync` is a waiter, never an `Error`; unselected `with-fallback` branches commit nacks; the append-only image is shared across actors.
W8 ✓	**Event-stream as program**	M–L (2–4)	W5	high	Landed as D29: image-resident append-only steps (`-10`), rollback to a step, and bookmarks absorbed as degenerate one-point events.
W9 ✓	**Linear ownership**	M–L (3–4)	W8 ✓	high	Landed as D30: per-run ownership over `EntryIdentity` in execution state; `own`/`borrow`/`consume`/`release`

ID	Item	Effort	Depends on	Risk	Acceptance criteria
					with named refusals; a spawned actor owns nothing; the image is never rewritten.
W10 ✓	**Multiple Dispatch** (D19)	M (2)	W3	medium	dispatch on sorts (not token strings); `executeWord` remains the lexical layer beneath. Landed as D19: `generic`/`method` words, image-resident generic functions (`-4`) and methods (`-5`) under `GenericFunctionsRoot`, type-first selection with a static exact-tuple fast path and `UnknownType` membership fallback, extension-subset specificity, and named ambiguity/no-method verdicts
W11 ✓	Tensor algebra beyond scalars	M	—	low	Landed as D60: arithmetic and comparison are elementwise over equal shapes; a scalar is the rank-0 case; a shape mismatch is a named `ShapeMismatch`; `readScalar` refuses a non-scalar instead of reading element 0.
W12 ✓	**The loop executes the compiler's output** (D9)	S–M (1)	W1, D7	low	one emitter with two domain bindings; two compiled artifacts (`PlayerSpecificationPristine`/ `PlayerSpecificationRefined`) instead of the in-process mirror; re-emit is byte-identical; `demo.fs`'s private emitter deleted; D7's outcome tests still hold; D9 flips to applied
W13 ✓	**SMT discharge: properties become provable** (D10)	M (1–2)	W12, D9	medium	`Microsoft.Z3` in-process; SMT encoding generated from the same parsed document; `Proved`/ `Falsified (model)`/`Unknown`/ `Unavailable` reported individually; `.smt2` obligation emitted and byte-pinned; differential validation against the compiled predicate (exhaustive on the demo universe + every returned model); the `.lean` stub and its tests retired; Hedgehog remains the fallback
W14 ✓	**Refiner on proved**	S–M (1)	W13	low	guard closure is the SMT query `pre ∧ guard ∧ ¬post` unsat (the

ID	Item	Effort	Depends on	Risk	Acceptance criteria
	guard closure (D10)				refinement strengthens the precondition; the earlier ¬guard note was inverted), not an exhaustive scan of the universe; the weakest *proved* guard is selected; provenance records the model and solver version; D7's enumerable-universe limit closes
W15 ✓	**Fixed-spec mode + acceptance gate** (D11)	M (1–2)	W13, W14	medium	`Mode = Exploratory
W16 ✓	**Verification type lattice** (D25)	S–M (1)	W14, W15	low	One lattice is the shared field/sort/coordinate/narrowing description for Z3, Hedgehog, and the refiner; non-integral coordinates and thresholds are refused before sampling; the record representations are opaque; `Refiner.refine` folds the document's existing refinements into coverage; the emitter refuses lattice/document divergence; suite 424 + 48. See `docs/type-lattice.typ`.

3.5 Done — W6, channels are image-resident endpoints

Status: done (applied as D20).

Goal. Implement W6 channels as image-resident communication endpoints with blocking as a distinct outcome, not an error.

What landed.

1. **`components/prelude/evaluator/channel.fs`** — four words: `channel`, `send`, `receive`, `close`. Each returns `Result<EvaluationState, EvaluationError>`; blocking halts the actor with a `ChannelWaiterType` payload, never an `Error`.
2. **`types.fs`** — new types: `Channel`, `ChannelOutcome`, `ChannelWaiterType`, and four `EvaluationError` cases: `ChannelNotFound`, `ChannelClosed`, `ChannelSendFailed`, `ChannelReceiveFailed`.
3. **`SchedulerState`** gains a `Channels: Map<EntryIdentity, Channel>` field; roots `initialSchedulerState` and `initializeSchedulerState` initialize it empty.
4. **Channel encoding** — a channel is a tensor-encoded record in the image (`ChannelTag = -6.0` reserved per D16); the scheduler's `Channels` map is the live registry. Persistence is free (channels serialize with the image).
5. **Test in `development/scale/tests.fs`** — `"channel creates a channel and pushes its ID"` replaces the three "not yet implemented" checks for `channel`/`send`/`receive`.

Acceptance. Channel ID is pushed as an `EntryIdentity` tensor; channels survive persistence (root re-threaded like `SetsRoot`); no host `Map` for channels.

Non-goals (deferred to W7 ChannelEvents). `choose`/`wrap`/`guard`/`with-fallback`, buffered channels, channel ownership transfer.

3.5.1 Done — D21, word metaobjects are image-resident definitions

Status: done (applied as D21).

Goal. Implement D16 Stage 2 word metaobjects: the vocabulary's `FsharpName` seam is replaced by a `Word` metaobject carrying token, form, category, active flag, and implementation reference. The dispatcher consults the word registry before the vocabulary.

What landed.

1. **`types.fs`** — `WordTag = -6.0` allocated (D16 registry); `Word` metaobject type with `Address`, `Token`, `Form`, `Category`, `IsActive`, `Implementation`; error cases `InvalidWordCategory`, `InvalidWordActive`.
2. **`components/prelude/evaluator/word.fs`** — `word` declaration word (`token form category active implementation word`); stack-based like `generic`/`method`; encoding/decoding helpers; `findByToken` lookup.
3. **`components/prelude/evaluator/evaluator.fs`** — dispatcher order updated: Built-ins → Word metaobjects → Generic functions → User words → Vocabulary.
4. **`types.fs`** — `WordsRoot: EntryIdentity option` added to `EvaluationState`; roots re-threaded.
5. **Tests in `development/scale/tests.fs`** — declaration, consultability, invalid category/active errors, latest-write-wins, persistence round-trip.

Dispatcher order (per D21):

1. Built-in cases (literals, quotations, control words)
2. **Word metaobjects** (new — image-resident, user-extensible)
3. Generic functions (`generic`/`method`)
4. User words (`UserWordsRoot`)
5. Vocabulary (`VocabularyRoot`)

Acceptance criteria (D17 invariant 2):

- A new word family needs no F# case.
- A protocol decision returns a verdict, never a silent false.
- The word metaobject round-trips through persistence.

Non-goals (deferred). Surface syntax for word declaration in the parser (currently only available via the `word` stack word); `choose`/`wrap`/`guard`/`with-fallback` for ChannelEvents.

3.5.2 Done — W7, ChannelEvents

Status: done (applied as D26).

Goal. Add ChannelEvents-style first-class events over the W6 channels while keeping a would-block a distinct outcome, not an error.

What landed.

1. **`components/prelude/evaluator/event.fs`** — seven words: `send-event`, `receive-event`, `choose`, `wrap`, `guard`, `with-fallback`, `sync`. Events are image-resident tensors tagged `EventTag = -8.0`; constructors push an address and only `sync` commits.
2. **Readiness.** `sync` selects the first ready branch. A channel `receive` is ready only when a message is queued; a false `guard` blocks; a `wrap` inherits its inner readiness. `WouldBlock` registers the actor in `SchedulerState.EventWaiters` and halts it; `send`/`close` wake every waiter, which restores `eventId sync` and retries. A blocked `sync` never produces an `Error`.
3. **Commit.** Channel actions run before `wrap` quotations are prepended to `Program`, so a wrapper sees the selected value. `choose` collects the fallback events of unselected `with-fallback` branches and commits those that are ready. `guard` evaluates its quotation against a discarded probe state under a reduction budget and refuses scheduler words.
4. **Shared image.** `switchTo` threads the live append-only image into the resumed actor. Otherwise a child that interns a message after its parent has written entries queues an `EntryIdentity` that aliases the wrong parent entry; the blocked-sync wakeup test pins this.
5. **Tests in `development/scale/tests.fs`** — send/receive events, choose selection and untouched unselected branches, wakeup after a later send, guard false/true, wrap, and `with-fallback` win/loss; five error-rendering cases.

Acceptance. `choose`/`wrap`/`guard`/`with-fallback` work over channels, blocking is not an error, non-event semantics are unchanged, the suite is 441 example checks plus 48 granular tests, and `just release` passes under NativeAOT.

Non-goals. Buffered channels, ownership transfer, and timeouts.

3.6 Done — W12, the loop runs the compiler's output

Status: done (applied as D9; see below for the record of what landed).

Goal. Make the refinement loop execute the emitter's *compiled* output, so the project's central claim ("the spec is the test suite") is true of the demo by construction. Previously the loop's confirmation step evaluated an in-process mirror (`demo.fs`'s private `propertyFor`/`preconditionOf`) and `demo.fs` carried a **second emitter** of its own — a two-sources-of-truth defect inside the product track itself. D9 settled the design; W12 landed it.

What landed.

1. **`emitter.fs` is the only renderer.** It takes a `DomainBinding` supplied as data: `ModuleName`, `SubjectType`, `VerbName`, `FieldName`, `FieldAccess`, `GeneratedRecord`, `DomainDefault`, `SourceOfTruth`, `Opens`, and an optional `LeanBinding`. `Emitter.selfContained` builds the golden binding; `demo.fs` builds the host-bound one (`HostBound`) over `DemoDomain`. Output stays framework-free (D5): a `(==>)` and a `: bool` predicate.
2. **`demo.fs`'s private `emitFSharp`/`emitLean` and its `propertyFor` mirror are deleted.** The loop routes through `emitter.fs`.
3. **Two compiled artifacts, both in `bases/scale-cli/scale.fsproj`** (after `demo-domain.fs`, before `refiner.fs`/`demo.fs`): `components/demo/demo/generated/PlayerSpecificationPristine.fs` (module `Scale.Generated.PlayerSpecificationPristine`, pristine document) and `components/demo/demo/generated/PlayerSpecificationRefined.fs` (module `Scale.Generated.PlayerSpecificationRefined`, refined document). Names are flat and distinct.

4. **The loop runs the compiled predicates.** `Refiner.falsify` runs a thin harness over `Scale.Generated.PlayerSpecificationPristine.``player canJump``; the confirmation run harnesses` Scale.Generated.PlayerSpecificationRefined.`player canJump`. Generators stay host inputs. The refiner, its diagnostic, and the recheck seed are unchanged.
5. **Pinned in `development/scale/tests.fs`:** the compiled initial predicate is falsified over the domain universe, the compiled refined predicate holds over it, the loop's shrunk counterexample flips between the two, and re-emission is byte-identical to each checked-in artifact.
6. **Landed in lockstep:** D9 flipped to applied, the ledger row and D7's first limit updated, README's "Where it stands" and this roadmap's state paragraph updated.

One correction the landing surfaced. F#'s `==>` binds tighter than `&&`, so a multi-clause precondition is parenthesized (`((player.Alive && player.Health > 1.0) ==> canJump player`). A single clause stays bare, so the pinned golden output is byte-unchanged.

A refiner input remains host-side, by design. The refiner's exhaustive search still needs the specification's precondition as a host function (it searches guards *beneath* the implication, which the compiled predicate cannot expose); that function is a refiner input like the generators, not a runtime check. The runtime check is the compiled predicate (D9 sub-decision 3).

Non-goals (out of W12). Wiring a Lean toolchain (still `sorry`); solving quantifier polarity or transition emission (D8); sampling-based refinement for large domains; a general "spec → program" compiler beyond the demonstration's one sentence, one field, one guard shape.

3.6 Done — W13, properties discharged by Z3

Outcome (landed). `domain-expression.fs` holds the implementation as one body with two backends; `smt.fs` encodes it and reports `Proved` / `Falsified (model)` / `Unknown` / `Unavailable`; `Emitter.emitSmt` writes a byte-pinned `.smt2` beside each compiled artifact; the `.lean` stub and its tests are gone. The demo's initial obligation is `Falsified (alive=true, health=1)` and the refined one is `Proved`; the suite was 146 checks at W13. W14 has replaced the refiner's exhaustive closure check with the proved query; the suite is now 153 checks.

Goal. Turn the property pipeline from *sampled* to *discharged*: the same parsed document that renders the F# predicate renders an SMT formula, and Z3 answers whether the property holds for **all** inputs in the theory — not just the ones Hedgehog happened to generate. D10 has the design and the reasoning; this is the implementation plan.

Why this and not a proof assistant. See D10: the property is a boolean formula over booleans and linear arithmetic, which is SMT's decidable core; a proof assistant imports a second ecosystem and a semantics translation whose faithfulness cannot be executed and cross-checked. The decisive device is **differential validation**: the F# predicate is already compiled and runnable (D9), so the encoding can be tested *against* it.

Verified preconditions (spike, in this sandbox). `Microsoft.Z3` 4.12.2 restores and runs in-process on `net10.0`; `/usr/bin/z3` 4.16.0 exists as an offline fallback; models come back structured. Do not conflate the two version numbers.

Steps (dependency order; do not reorder):

1. **Add `Microsoft.Z3` 4.12.2** to `bases/scale-cli/scale.fsproj`. Note: the sandbox has had no network by default in past sessions; today it restores. If a future environment cannot, the CLI backend below is the fallback — do not let a missing package silently delete the feature.
2. **New `smt.fs`** (after `scale-file.fs`, before `emitter.fs`, so the emitter can project the obligation text from the same encoding):

- encode a `ParsedSentence` + `ScaleDocument` as Z3 terms: `Bool` consts for boolean fields, `Int` for numeric fields that are integer-valued by construction, guards as arithmetic atoms, `==>` as an implication;
- **encode the implementation under test, not just the property.** If the verb is an uninterpreted symbol, `pre ⇒ verb(p)` is satisfiable by construction and the "counterexample" is meaningless. The verb's body must be part of the formula. The low-debt way to guarantee that is a tiny **domain expression** (`DomainExpression`: bool/int leaves, comparisons, `and`/ `or`/`not`, record field access) with **one body and two backends** — an interpreter that Hedgehog executes, and an encoder that Z3 discharges. Do not hand-write a second implementation for the solver; that is the translation debt D10 exists to avoid;
- `discharge` returns a four-way outcome: `Proved` (`pre ∧ ¬post` unsat), `Falsified` (sat + the model), `Unknown`, `Unavailable`;
- the `.smt2` text is `expr.ToString()` (Z3's printer) — **one source**, not a hand-written second rendering;
- add the adequacy queries: `satisfiable pre` (unsat ⟹ vacuous property) and `satisfiable (not post)` (unsat ⟹ tautological postcondition), reported as warnings, not passes.

3. **Retarget the obligation slot.** D9's `LeanBinding` becomes an `ObligationBinding`; the emitted artifact becomes `components/demo/demo/generated/PlayerSpecificationRefined.smt2` in place of `PlayerSpecificationRefined.lean`; the "Lean stub / `sorry`" tests are retired in the same commit that removes the artifact (never leave a test asserting a file that no longer exists).
4. **Report outcomes honestly.** Extend the suite tally so `Proved` / `Falsified` / `Unknown` / `Unavailable` are counted individually (D6's style). `Falsified` fails the process; `Unknown`/ `Unavailable` print an explicit note like `GaveUp` and must **never** read as a pass. Hedgehog remains the fallback evidence and the fast bug-finder.
5. **Differential validation (the acceptance core).** For every player in `DemoDomain.universe`, the SMT encoding's truth value equals the compiled predicate's; for every model Z3 returns, feeding the model's values to the compiled predicate genuinely falsifies it; and the `.smt2` artifact is pinned byte-identical. A silent mistranslation must show up as a test failure, not as a wrong theorem.
6. **Land in lockstep.** D10 → applied; ledger row (already retitled to SMT); README and demo README; roadmap counts and state. Update the test totals in the same commit.

Non-goals (explicitly out of W13). Moving the refiner onto proved guard closure (**W14** — the payoff, but a separate change); verifying the evaluator (Boogie + Z3, a later rung); Fable/z3-wasm (Q6); re-exporting Lean.

Risks. Medium. Theory alignment is the real one: `Health` is a `double` in F# but integer-valued by construction, so `Int` is exact today — widening the domain means widening the theory and naming the NaN/infinity caveat. Second: `solver.Check()` returning `Unknown` must be handled, not thrown. Third: the native library must load in whatever CI exists; the CLI fallback is the answer, and the outcome must be reported rather than skipped.

3.7 Done — W14, the refiner's closure is proved

Outcome (landed). `Refiner.refine` no longer decides closure by scanning the universe. It takes a `Refiner.Closure` oracle; `demo.fs` answers it with `Smt.discharge` on the document's conditions plus the candidate guard and the implementation's body, and accepts the guard only when the obligation is `Proved` **and** `Smt.satisfiablePre` says the guarded precondition is satisfied. The finite universe is still used for three smaller jobs: proposing single-field thresholds, rejecting a proved guard that has no witness in the demo, and ranking the weakest proved guard by coverage. It is no longer the soundness boundary.

The selected candidate carries structured provenance: the native solver's version (`4.16.0.0` here, from the loaded `libz3`) and the Z3 model of the unrefined obligation (`alive=true, health=1`). The demo prints both, and `components/demo/demo/player.scale` persists them as `origin: z3 <version>` and `counterexample: <model>`. The demo still chooses `health > 1` (9 of 21 inputs in scope), and the refined obligation is `Proved`; the suite was 153 checks at W14 and is 186 after W15 (197 after the review fixes).

Sign correction. The query that closes the class is `pre ∧ guard ∧ ¬post` unsat, because the refinement *strengthens* the precondition. The earlier roadmap row wrote `pre ∧ ¬guard ∧ ¬post`, which proves that every failure is outside the guard and would select the complement — it admits exactly the inputs that fail. That was an inverted sign in the plan, not an implementation choice; the row and this section now record the correct query.

Why D7's universe limit closes. Closure is a proof over the theory, not a `List.forall` over the demonstration's players. A larger domain still needs a richer *candidate generator* and a coverage heuristic, but it no longer needs an exhaustive closure scan, and no finite sample can make a guard "proved".

3.8 Done — W15, fixed-spec mode and the acceptance gate

Status: done (applied as D11). `repair.fs` defines `Mode`, `Witness`, `Target`, `RepairBundle`, `Patch`, `Verdict`, and `CorpusEntry`. `accept` enforces the four rules and rejects `Unknown`/`Unavailable` (neither is a pass); `appendInMode` makes the spec writer unreachable by rule; and `bundleJson`/`writeBundle` plus the corpus reader/writer persist the interface. The demo reports `mode: exploratory` and routes its only spec append through `appendInMode`; `witnesses/player.json` is the checked-in append-only corpus. The suite was 197 checks at W15.

Goal. Turn W13/W14's verifier into an unattended acceptance loop for an implementation whose specification the developer holds fixed (D11): a witness is a defect report, the loop never edits the spec, and a candidate patch is accepted only by the four-rule gate.

The mode switch. `Repair.Mode = Exploratory | FixedSpecification` is a value carried on `LoopReport`, not an inference from a witness. In `Exploratory`, D7's refiner appends refinements as today. In `FixedSpecification`, the spec-append path is disabled **by rule** — an attempted spec edit is a failed patch, not a warning — and a witness is routed to the repair bundle.

The repair bundle (the unit an AI or a human patches against) is machine-readable data, not prose. It must contain: the property and its `ParsedSentence`, the spec hash, the shrunk Hedgehog witness, the Z3 model, the encoded implementation that produced it (the `DomainExpression`/SMT term), the differential result, and the source location. That bundle is the contract between the loop and the generator — the loop's job is to make failure *actionable*, not to write code.

The gate (`accept : Patch -> Verdict`) re-runs, on the patched implementation: (1) discharge of the target property; (2) the full witness corpus (no regression); (3) a spec-hash check (unchanged); (4) the differential checks (encoding still agrees with the compiled predicate). `Verdict = Accepted | Rejected of reason`; only `Accepted` closes the loop.

Steps.

1. **`DomainExpression`-backed implementation** (landed with W13, reused here): the verb body is one value, interpreted for execution and encoded for Z3. W15's `Patch` replaces that body (`Body`) and carries an optional `SpecEdit`, so candidate patches are values, not free text, and can be gated deterministically.

2. **`repair.fs`** (landed): `Witness`, `RepairBundle`, `Patch`, `Verdict`, `accept`, `makeBundle`, `appendInMode`, and the JSON persistence functions. They are pure functions over the existing `Smt`/`Refiner` results; the gate adds no solver beyond `Smt.discharge`.
3. **Mode plumbing** (landed): `Repair.Mode` is carried on `LoopReport` and printed by the demo. The demo stays `Exploratory`; `appendInMode FixedSpecification` refuses to write, and the suite asserts that refusal.
4. **Witness corpus persistence** (landed): `witnesses/player.json` is the checked-in, append-only corpus. The reader accepts the checked-in object form and the writer's `{ name, value }` array form; `writeCorpus`/`readCorpus` round-trip including refinements.
5. **Reporting**: the demo prints its mode and the existing `Proved`/`Falsified`/`Unknown` outcomes; the fixed-spec gate's verdict and bundle are exercised by the suite. The CLI run landed as D28: `scale --fixed-spec` writes the bundle to `artifacts/repair-bundle.json`, never edits the specification, and exits non-zero when the fixed specification is falsified.
6. **Tests** (landed): a witness yields a bundle containing the model; a spec edit is rejected in `FixedSpecification`; a patched implementation that satisfies the spec is accepted; a patch that breaks a previously passing property is rejected; an `Unavailable` target is rejected rather than counted as accepted; an inconsistent witness is caught by the differential check.

Non-goals. Proving the evaluator (Boogie rung); an autonomous code generator (the bundle is the interface — the generator is outside the loop); Fable/z3-wasm; Lean export.

Risk. Medium: the bundle's shape is an interface, so it will be hard to change once a generator consumes it; keep it versioned from the first commit. The gate's "no regression" needs a corpus that is committed, or acceptance is not reproducible.

3.8.1 Done — W16, the verification type lattice (D25)

Status: done (applied as D25). `type-lattice.fs` is the one description of the verification domain: each field has a name, a `DomainExpression.FieldType`, optional finite coordinates, and a narrowing guard list. `Smt`, Hedgehog, and the refiner read the same lattice; `DomainExpression.integerValue` is the one Int32/QF_LIA boundary, so sampling cannot outrun the encoding. The record representations are private and the public constructors are `create`, `ofFieldTypes`, `applyGuard`, and `withGuards`.

`Refiner.refine` folds the document's existing refinements into the lattice before enumerating candidates, so coverage and the returned lattice describe the current region. The emitter refuses a lattice narrowing the document does not carry, so the F# and SMT artifacts cannot state different preconditions. The demo's initial binding uses the current lattice. The suite is 541 example checks and 53 granular tests at D33. Property synthesis remains the documented dual in `docs/type-lattice.typ`, not an implementation. The Transition corpus refusal is pinned so the deferred pattern cannot silently acquire Capability semantics.

Goal. Close the seam between the existing `FieldType` record, the SMT encoding, the Hedgehog generators, and the refiner: one coordinate system, validated before any reader can disagree about what a field may hold.

3.8.2 Done — D27, the trust boundaries refuse instead of assuming

Status: done (applied as D27).

Goal. A review of the whole tree found nine places where the implementation could contradict a guarantee the ledger claims. Most of them sat at a boundary the project treats as load-bearing: the refiner's verdict, the `.scale` document, the emitter's threshold, the channel's state, the solver's

answers, the persistence reader, and the release tooling. Each is now refused explicitly and pinned by a regression test; the record is D27.

What landed.

1. **Inconclusive is not a pass.** `Refiner.Falsification` gains `Inconclusive`; Hedgehog's `GaveUp` maps to it, and the demo reports it.
2. **The document escapes its free text.** A `.scale` field can no longer inject a directive line, and the document round-trips exactly.
3. **A threshold is emitted exactly or refused.** The emitter no longer rounds `1.25` to `1.2`, and a non-integral threshold is refused rather than approximated.
4. **A channel's state is image-resident and append-only.** The identity entry is written once; the open flag and the queued messages are appended state entries. Closing appends instead of rewriting, a queued message survives persistence, and a terminated actor leaves the waiter lists so it cannot swallow a message.
5. **A set literal refuses actor words.** `SetLiteralSchedulerWord` names the refusal.
6. **An inactive word metaobject is not callable.**
7. **The boundaries validate their input.** Tensor append, `term` arity, and the persistence reader; persistence also writes atomically.
8. **The solver's answers are checked.** Assignments must be complete, single, and well-sorted; a model value outside the domain's range is `Unavailable`; an undecided adequacy query is an `Error`.
9. **The release tooling cannot delete outside the artifact tree.** The RID allowlist is closed, and the release workflow runs the complete gate before it publishes anything.

Also. The random program generator now draws from the whole word surface, so the Railway-totality and determinism laws cover the scheduler, channels, events, sorts, unification, dispatch, and word metaobjects; and the dispatch property asserts that the probe really is a member of the declared sort.

Suite. 471 example checks and 48 granular tests.

Non-goals. No change to the language's semantics: a set literal still evaluates its body on a fresh stack, a channel is still a stack word, and the refiner still searches for the weakest proved guard.

3.8.3 Done — D28, the thin follow-ons

Status: done (applied as D28).

Goal. Close the two items W15 named as thin follow-ons: a CLI fixed-spec run that writes the repair bundle and fails on a falsified target, and emission for a subject that declares no conditions.

What landed.

1. **`scale --fixed-spec`.** The run mode lives in `cli.fs`; `Demo.runFixedSpec` holds the specification fixed (no spec edit, no artifact re-emission), turns a Hedgehog witness into a `Repair.makeBundle` bundle at `artifacts/repair-bundle.json`, and reports it. Exit codes: `0` holds, `1` falsified or inconclusive, `2` the run cannot start. Accepting a patch stays `Repair.accept` in-process — the CLI is the reporting path.
2. **A subject with no conditions.** "Every player can jump." parses with no conditions; the subject type is now a marker union (`type Player = Player`) instead of an invalid empty record, and the pinned artifact `generated/no-condition-sample.fs` is compiled into the project.
3. **The subject type carries every named field.** Conditions and refinement guards are deduplicated by field, so a repeated condition emits one field and a refinement-only field is declared. One field with two sorts is refused.

Suite. 480 example checks and 48 granular tests.

Non-goals. No new surface word, no change to the fixed-spec acceptance rules, and no autonomous patch generation: the bundle remains the interface.

3.8.4 Done — D29, the event-stream as the program (W8)

Status: done (applied as D29).

Goal. History including program-counter metadata, rollback to a step, and bookmarks absorbed as a degenerate case rather than a parallel mechanism.

What landed.

1. **Steps.** `components/prelude/evaluator/history.fs` records a step before every token the top-level loop executes: `[StepTag; location; baseline; position; tokenId; stackId; programId]`, with `StepTag = -10.0`. The stack and program are image vectors of addresses, so the append-only image keeps them valid.
2. **Rollback.** `rollbackTo` restores the stack, the program, and the program-counter metadata, and re-points the history root at the steps before the target (a new vector). The image is never rewritten; later steps stay, unreachable. `history` pushes the steps before itself; `rollback` restores a step, and an out-of-range index is a named refusal.
3. **Bookmarks absorbed.** `bookmark` records a step and pushes `[location; baseline; position; stepId]`; `jump` restores through `rollbackToStep`. There is one rollback mechanism, not two.
4. **Boundary.** The history covers the top-level `run` loop; a set literal's inner interpreter and a `guard` probe are expressions, not steps. The property generator includes `history`; `rollback` is excluded because a program that rolls back and re-runs the same suffix is a loop by construction.

Also. This landing added the granular integration and Verify snapshot suites: the fixed-spec CLI, the refinement loop end to end, and snapshots of the no-condition artifact, the repair-bundle wire format, and the escaped `.scale` document.

Suite. 495 example checks and 53 granular tests.

Non-goals. Preemptive or timeout-driven history, history compaction, and a reflective evaluator. W9 (linear ownership) is the next head of Track C.

3.8.5 Done — D30, linear ownership (W9)

Status: done (applied as D30).

Goal. Per D1 (execution state): per-run ownership over `EntryIdentity`; consume/borrow checks; diagnostics; ownership never serialized.

What landed.

1. **State.** `EvaluationState.Ownership` holds the owners (value address ⟶ owning actor) and the consumed addresses. It is execution state: neither a persistence section nor an image entry.
2. **Words.** `own` takes ownership (idempotent for the actor's own value), `borrow` is a checked read, `consume` ends the value's life for the run, and `release` gives ownership up. `NotOwned`, `OwnedByAnother`, and `ValueConsumed` are the named refusals.
3. **Actors.** A spawned actor owns nothing; ownership does not transfer implicitly.
4. **Boundary.** The checks are dynamic and live at the ownership words: the append-only image is never rewritten, and other words can still read an entry. Ownership is a discipline, not a static type system.

Suite. 505 example checks and 53 granular tests.

Non-goals. Ownership transfer, borrow counting or regions, static linear types, and serializing ownership (D1 keeps it in execution state).

3.8.6 Done — D31, predicate sorts (D16/D17 stage 3)

Status: done (applied as D31).

Goal. A new sort constructor needs no F#; membership returns a verdict.

What landed.

1. **Encoding.** A predicate sort is `[PredicateSortTag; name; quotation; member…]` (`-11.0`), image-resident and append-only; the members are the base extension and the quotation narrows it.
2. **Surface.** `base { predicate } :Name predicate-seal`; the predicate runs with the candidate on the stack and must leave a numeric verdict (nonzero is a member).
3. **Verdict.** `Member`/`NotMember`/`Undecided reason`; `is`, `member`, and `subset` read it, and `subset` is decided memberwise and propagates `Undecided`. An erroring predicate, an exhausted budget, a scheduler word, or no verdict is `Undecided`, never a silent false.
4. **Boundary.** `union`, `intersect`, `except`, `same-shape`, `merge`, `complement`, `and`, `or`, `nor`, and `admit` refuse to enumerate a predicate sort's extension (`ValueSetUndecided`); `seal`/`unname` preserve the predicate; a predicate sort is not an event branch list.

Suite. 515 example checks and 53 granular tests.

Non-goals. SMT encoding of predicate sorts, quantified predicates, and enumeration of a predicate sort. The verification lattice (D25) is a separate coordinate system and is unchanged.

3.8.7 Done — D32, classification

Status: done (applied as D32).

Goal. One membership verdict, one class query, no silent classification.

What landed.

1. **One verdict in dispatch.** `inferType` and `matchesSpecializer` use `memberOf`, so predicate sorts participate instead of being read by their base extension. An undecidable predicate is `DispatchUndecided`; specificity compares verdict-aware subsets.
2. **`class-of`.** `[ClassTag; status; payload…]` with Unknown/Class/Ambiguous/Undecided/Metaobject. `classify` is the shared query; the word and the dispatcher cannot disagree.
3. **Entry kinds.** Derived from tags: word, generic, method, channel, channel state, event, step, predicate sort. Finite sorts stay root-identified; the query stays total without a per-tensor field.
4. **Boundary.** The verification lattice is a field-coordinate system, not a value-class system. The shared thing is the verdict vocabulary and the class query, not the representation (D32 records why a structural bridge would be a category error).

Suite. 532 example checks and 53 granular tests.

Next. The guard metaproperty and comprehension syntax remain the MOP's open items; existential `some` remains the named smaller item.

3.8.8 Done — D33, existential `some`

Status: done (applied as D33).

What landed. `some` projects to `Possibility`. The F# artifact is a witness predicate `pre && post ( some )`; the SMT obligation is `pre ∧ post` satisfiability, with `Witnessed model` / `NoWitness`.

Fixed-spec acceptance and the witness corpus refuse possibilities by name. The compiled sample is `generated/possibility-sample.fs`.

Suite. 541 example checks and 53 granular tests.

Non-goals. The refiner (no counterexample to shrink); possibility corpus entries; a possibility harness in the demo.

Next. The filtering word for comprehensions (D34), then the MOP guard metaproperty.

3.8.9 Done — D34, the filtering word

Status: done (applied as D34).

What landed. `sort { predicate } filter` narrows a sort to the members its predicate accepts, using the one membership verdict: base membership first, then the filter predicate. An undecidable predicate is `ValueSetUndecided`, never a silent drop. The result is an anonymous structural sort. This is the filtering word comprehension syntax needs; the `[ ?x : … ]` surface remains deferred because it still needs a domain decision.

Suite. 546 example checks and 53 granular tests.

Next. The MOP guard metaproperty; then the comprehension surface once the domain/variable form is decided.

3.8.10 Done — D35, guard metaobjects

Status: done (applied as D35).

What landed. Guards are image-resident definitions `[GuardTag; name; quotation]`: `guard-define` declares one, `guard-holds` runs its quotation through the shared probe and answers 1/0, with `GuardUndecided` and `NotAGuard` as named refusals. `entryKind`/`class-of` recognize guards (kind 9). The probe moved to `components/prelude/evaluator/probe.fs` and predicate sorts use it too. The verification pipeline's typed `ScaleFile.Guard` remains the SMT/emitter projection; the lattice integration is the named remainder.

Suite. 552 example checks and 53 granular tests. The kernel-inventory headline was corrected to the full dispatcher count (78).

Next. The guard-lattice bridge landed as D41; then the full cell system.

3.8.11 Done — D36, sort constructors

Status: done (applied as D36).

What landed. A constructor is an image-resident metaobject `[ConstructorTag; name; quotation]`: `"name" { predicate } constructor` declares one, and `sort constructor "name" constructor-seal` builds a named sort whose base extension is the operand and whose narrowing runs the constructor quotation through the shared probe. `member`/`subset` resolve either a raw predicate quotation (D31) or a constructor reference (D36). `entryKind`/`class-of` recognize constructors (kind 10). An undecidable constructor is `ValueSetUndecided`; a non-constructor is `NotAConstructor`. The set operations remain kernel defaults and refuse a constructor sort rather than approximate it.

Suite. 562 example checks and 53 granular tests. The kernel-inventory headline moves from 78 to 80 explicit word cases.

Next. The set-operation protocol methods landed as D39 and the guard-lattice bridge as D41; then the full cell system.

3.8.12 Done — D37, value-set naming

Status: done (applied as D37).

What landed. The evaluator type is `EvaluatorValueSets.ValueSet` in `components/prelude/evaluator/value-sets.fs`; a field's domain type is `FieldType`; the execution state uses `SetsRoot` and `DeclarationsRoot`; the operations are `readSet`, `writeSet`, `pushSet`, `setsCount`, `derivedSet`, and `setLabel`; the error cases are `ValueSetMismatch`, `ValueSetNotClosed`, and `ValueSetUndecided`. `Sort`, `FieldSort`, `ModelsRoot`, `SortsRoot`, `readModel`, `writeModel`, and `pushModel` are gone from the code. Prose may still say *sort* as the type-theory term; it is defined once as a named value set.

Why. `Sort` read as ordering and named three different things; `Model` named a subset, a storage root, and an SMT assignment; `FieldSort` carried the same overload. The code now says what the thing is, without changing the surface language.

Suite. 562 example checks and 53 granular tests, unchanged.

Next. Qualify the remaining overloaded names (`Event`, `Guard`, `Smt.Model`) and keep the surface words stable until a decision record changes them.

3.8.13 Done — D38, identity and ChannelEvents

Status: done (applied as D38).

What landed. `EntryIdentity` is now `EntryIdentity`; `Identity`, `identity`, and `identities` are `Identity`, `identity`, and `identities`. The concurrency pattern called `ChannelEvents` is named **ChannelEvents**: first-class synchronous events over channels (`send-event`, `receive-event`, `choose`, `wrap`, `guard`, `with-fallback`, `sync`). The mapping and the naming philosophy live at the top of `tools/rename-names.fsx`.

Why. Identity is the full word for the concept; Identity was an abbreviation. ChannelEvents describes the pattern without borrowing an acronym from another community.

Suite. 562 example checks and 53 granular tests.

Next. The guard-lattice integration; set-operation methods landed as D39. Then the full cell system.

3.8.14 Done — D39, set-operation protocol methods

Status: done (applied as D39).

What landed. `member`, `subset`, `union`, `intersect`, `except`, and `complement` are image-resident generic functions seeded by `initial`. Each has a default method specialized on the empty `set` declaration; the method body calls the kernel primitive (`set-member`, `set-subset`, `set-union`, `set-intersect`, `set-except`, `set-complement`). A set entry is classified as `set` by dispatch. Replacing a method is now an image change.

Why. D17 stage 3 says the sort metaobject carries its protocol methods. The six operations were explicit `executeWord` cases; now the operation is selected from the image, and the default is only a method body.

Boundary. The primitives still implement finite, predicate, and constructor policy in F#. A constructor sort still refuses enumeration. A non-set operand now has no applicable method rather than `NotASet`.

Suite. 564 example checks and 53 granular tests.

Next. The guard-lattice integration and the full cell system.

3.8.15 Done — D40, speech-first names and the mascot

Status: done (applied as D40).

What landed. The product is Scale; Lia is the mascot, documented in the index and glossary only. `withNack`/`nack` became `with-fallback`/fallback, `Reifier`/`reification` became `Refiner`/refinement, `snoc` became `appendElement`, `BAG` became Vocabulary, `F# name` became Implementation name, `FixedSpec` became `FixedSpecification`, and our prose uses "apply the substitution" instead of zonk. Generated artifacts now say `Scale.Generated` and "Generated by Scale".

Why. The names are spoken aloud; ordinary words survive speech-to-text. SMT, AOT, CLI, MCP, LSP, JSON, and Z3 stay because they are established names. `Kitten.Zonk` stays because it is an external API citation.

Suite. 564 example checks and 53 granular tests.

Next. The guard-lattice integration landed as D41; then the full cell system.

3.8.16 Done — D41, the guard-lattice bridge

Status: done (applied as D41).

Goal. Close D35's named remainder: a document refinement can name an image-resident guard metaobject, and the verification lattice evaluates it through the same guard the runtime uses, instead of keeping a second guard semantics.

What landed.

1. **A named refinement.** `ScaleFile.RefinementGuard.NamedGuard of field * name`; the `.scale` surface is `refinement: <field> guard <name>`.
2. **A resolver seam.** `TypeLattice.GuardVerdict` is `GuardHolds | GuardDoesNotHold | GuardUndecided of reason`, and `GuardResolver` is `name -> FieldValue -> GuardVerdict`. The lattice gains resolver-aware `coordinatesAfterNarrowing`, `assignments`, `universe`, `applyGuard`, `withGuards`, and generator functions; the existing names remain the typed-only path and refuse a named guard by name.
3. **The image supplies the resolver.** `EvaluatorRuntimeGuard.findByName` scans the append-only image for the guard metaobject, `evaluateByName` runs its quotation in the shared probe, and `resolver` maps the probe result to a verdict. A missing or undecidable guard is `GuardUndecided`, never a silent false.
4. **The proof boundary stays honest.** A quotation has no SMT encoding, so `Smt.refinementGuardToBooleanExpression` returns an `Error` and the emitter refuses a named guard. A named refinement therefore drives the finite generators and coverage but cannot be discharged; it is never silently omitted from an obligation.
5. **Pinned** by twelve checks: the document round-trip, the image-backed resolver (hold, non-hold, missing), the named guard and the equivalent typed guard agreeing on every lattice member, the typed-only path refusing a named guard, an undecidable guard being a named refusal, and the SMT refusal.

Suite. 576 example checks and 53 granular tests.

Non-goals. A symbolic interpreter for guard quotations (the SMT side stays `Unavailable` for a named guard), refiner synthesis of named guards, and the full cell system.

3.9 Done — decision tables (D13; promoted as D73)

Status: promoted as D73 (see 3.46). The prepared layer is kept as the record; the host-level binding and per-row emitter are landed.

What landed. `decision-table.fs` (compiled after `refiner.fs`), with:

- `Table<'outcome>` — a name, the input field sorts, a hit policy, and ordered `Row<'outcome>` values (a guard conjunction plus an outcome);
- `HitPolicy = Unique | First | Priority`;
- `check` — Z3 proofs of **completeness** ($\neg(g_1 \vee \ldots \vee g_n)$ unsatisfiable) and, under `Unique`, **disjointness** ($g_i \wedge g_j$ unsatisfiable), each failure carrying a concrete `Smt.Model` witness;
- `select` — the pure runtime answer, built on `Refiner.evaluateGuard`, with `Selected`/`NoMatch`/`Ambiguous` so an unchecked `Unique` table never picks silently;
- three distinct failure modes: `Unsound` (witnessed), `Undecided` (solver gave up, never read as sound), and `Error` (unencodable).

To make that possible, `smt.fs` gained `DomainEncoding` (`createDomain`) — the field world without an implementation body — plus `guardToBooleanExpression` and `satisfiable`. `create` is now the composition of `createDomain` and body encoding, and it disposes the context when the body fails to encode (a leak on the error path is closed).

Why it stays opt-in. Nothing on the production path references it: not the parser, vocabulary, evaluator, or demo. `development/scale/tests.fs` is the only caller. A domain opts in by constructing a table and calling `check`, `select`, and `DecisionTableEmitter.emit`; a domain that never does is bit-for-bit unaffected. No new words and no parser or vocabulary change.

What it is not.

- It does **not** generate property tests from nothing. It *scaffolds* one obligation per row (`guards ∧ ¬body`) and one completeness obligation. The invariant, the generator, and the body stay with the domain or the human.
- It is **not** a machine-learning decision model. A classifier's output is a probability, which is neither a proof nor a witness, so it cannot enter the proof-or-witness trust boundary. See D13 for why that is a structural exclusion rather than a delay.
- The Basic English table surface is D74; the host-level promotion is D73. `Transition` is reachable again through D55 and is read as a guarded effect; a table does not depend on it.

Promotion (D73). `TableBinding` supplies one body per outcome; `obligationFor`/`obligations` expose each row as an `Smt.Obligation`; and `DecisionTableEmitter.emit` renders one property module per row with the row's guards as the precondition and `rowN` as the body the host supplies. The completeness proof remains `DecisionTable.check`, and the two-row demo is pinned in `snapshots/decision-table.verified.txt`. The emitted artifact assumes the host supplies `rowN`; it is not self-contained. D74 adds the Basic English table surface and the diagnostic sink. See `docs/decision-models.typ`, D73, and D74.

TODO: Wire the SMT/discourse detectors; the CLI table mode is `--table <path>`. Character offsets are D76.

3.10 Done — W3, sorts are a pair

Status: done (applied as D14). `components/prelude/evaluator/value-sets.fs` owns the image-resident sort registry; `components/prelude/evaluator/vivid.fs` keeps the model words and now delegates to it. A model entry is `[nameValue; member...]` with `-1` for anonymous; a second root (`DeclarationsRoot`) holds declared domains. Both roots are execution state (D4) and are re-threaded across persistence.

What landed.

1. **Literal sets.** `[ … ]` evaluates its body on a fresh stack and captures the values; it is an extension literal, not a comprehension. `[ ]` is the empty set; `[ 1 2 add ]` is `{3}`. The tokenizer splits `[`, `]`, and `:Name` out of a word, so `[1 2]:s` works without spaces.
2. **Structural words.** `subset`, `union`, `intersect`, `except`, `member`, `same-shape`; all compare by extension (`tensorsEqual`), and literals and unions deduplicate on that equality.
3. **Nominal words.** `S :Name seal`, `S unname`, `same-name`, `sort-of`; `:Name` references an existing seal. The surface is postfix because a stack language cannot carry a pending prefix seal without host state the level model forbids; the design note is corrected to match.
4. **The closed universe.** `complement` reads the declared domain, not the latest model, so a subset sealed under an existing name complements against the full domain. `merge` is the seal-preserving union; `ValueSetMismatch` and `ValueSetNotClosed` name the two refusals. The ternary `nor` is deliberately unchanged: a binary default-universe `nor` is ambiguous on a stack, so D3's line closes with a distinct unary word instead of an arity change.
5. **`model` is sort-aware.** `model "s"` declares `s` with an empty domain, `admit` grows both the model and the domain, and the derived `and`/`or`/`nor` models are declared too. Existing behavior and diagnostics are unchanged; all earlier tests and properties pass.
6. **Pinned in `development/scale/tests.fs` and `properties.fs`.** Thirty-two example checks (literal evaluation, empty set, membership, subset, union/intersect/except, both equalities, seal/unname, latest-write, `sort-of`, `merge`, `complement`, the three named refusals, both literal syntax errors, tokenizer splitting, persistence with both roots re-threaded) plus one property: union/intersect/except agree with ∪, ∩, and \ elementwise. Suite at W3: **245 checks (230 examples + 15 properties)**.

Deferred, explicitly. Comprehension syntax (`[ ?x : ?x > 1 ]`) is not landed as D34; the `[ ?x : … ]` surface remains deferred because it needs a domain decision. The decision-table layer still uses `DomainExpression.FieldType`, not the sort record. The `model` sugar remains name-as-sort: it declares a sort, but the pair is read through the new words.

3.11 Done — W4, unification is a real pillar

Status: done (applied as D15). `components/prelude/evaluator/environment.fs` now owns a first-order structural unifier, not a one-way assignment. The substitution is still `BindingsRoot`; terms are image entries, so there is no new root and no host term state.

What landed.

1. **Term encoding.** A variable is an interned `?name` text entry; an atom is any other entry; a compound is `[TermTag; head; argument…]` with `TermTag = -2.0`; a typed term is `[TypedTag; sortName; term]` with `TypedTag = -3.0`. `ValueSet.decode` now accepts only `-1` as the anonymous marker, so a term is never read as an anonymous sort.
2. **Unification.** `walk` resolves substitutions, `occursIn` rejects direct and indirect cycles, `unifyTerms` unifies variables, atoms, compounds, and typed terms, and `resolveTerm` applies the substitution recursively. Two unbound variables are aliased from the higher `EntryIdentity` to the lower one, so the result is independent of operand order. A failed match is transactional: it restores the pre-call root and image rather than leaving a partial binding.
3. **Nominal rule.** Two typed terms unify only when their sort **names** are equal; the extension is not consulted. A bare variable may bind to a typed term and thereby inherit its sort, but a typed term does not unify with an untyped non-variable. That is the W3 sort record doing exactly what D12 promised: W4 reads `N`, W10 reads `X` (landed as D19).
4. **Surface.** `term` builds `head arg… arity`; `typed` attaches `:Name`; `resolve` exposes substitution; `lookup` now resolves chains. `typed` refuses an undeclared name with `UnknownModel`, and retyping under a different name is `ValueSetMismatch`; invalid term arity is `InvalidTermArity`.

5. **Pinned.** Twenty-four examples plus two Hedgehog properties (variable-chain resolution and occurs-check at random depth). Suite: **271 checks (254 examples + 17 properties)**.

What the neighbours settled. Kitten's `Unify`/`Zonk`/`Occurrences` gave the discipline (resolve before matching, check occurrence on the resolved term, commute variable cases); Cat's `Unifier` gave the deterministic-representative idea (Scale uses the lower `EntryIdentity` in a persistent association vector rather than mutable union-find); Factor's stack-checker `unify` is branch/stack-effect unification and its runtime polymorphism is generic dispatch, so it stayed out of the value unifier; Joy has no variables at all. Prolog/miniKanren supplied the reference shape (`walk`, `occurs`, `bind`, recursive arguments) without the search or trail.

Deferred, explicitly. No higher-order unification, rational/cyclic trees, attributed variables, goal search, cut, or general type inference. The sentence layer is not a theorem prover, and polymorphism is W10's dispatch on the extension (landed as D19). See D15 for the full argument.

3.12 Prepared — the Metaobject Protocol (D16, D17)

Status: applied through D51. The generic-function and method substrate landed as D19 (W10), word metaobjects as D21, predicate sorts as D31, the guard runtime protocol as D35, sort constructors as D36, the set-operation protocol methods as D39, the guard's verification-lattice bridge as D41, the full cell system as D43–D50, and control refinement as D51. Every D17 stage is now applied; the cell-system layer is no longer proposed. The baseline is `docs/kernel-inventory.typ`.

The thesis. The language's own extension points — words, sorts, generic functions and methods, guard evaluators, control forms, and protocol failures — become image-resident, tensor-encoded metaobjects with a protocol and default methods. Adding behavior means adding a method in the image, not a case in F#. This is the Metaobject Protocol as *open implementation* not a reflective tower and not a wholesale CLOS clone.

What already fits. The vocabulary and sort registries are image-resident; `IsActive` is a meta-level flag; `DecisionTable` is already a method set without bodies (`Priority` is specificity, `Ambiguous` is incomparable methods, `check` is coverage and disjointness); `Smt.Outcome` and `DecisionTable.Verdict` already report `Undecided` rather than a silent false. D12′s pair is the class metaobject, D13′s table is the method-set prototype, and D14′s registry is the metaobject table. D19 (W10) lands generic functions and methods as image-resident metaobjects under `GenericFunctionsRoot`, so adding a method needs no F#. D43–D50 land classes, slots, instances, precedence, method combination, metaclasses, closures, conditions, and restarts. D51 lands control forms as `Word` metaobjects whose implementations are either default kernel primitives or code cells.

What remains. No stage of D17 remains. The default method bodies for set operations, arithmetic, stack shuffling, and control primitives stay in the kernel by design; the protocol makes the dispatch and replacement image-resident, not every primitive body. Replacing a control form with a code cell needs no F#, as D51′s tests show. The optional DSPy-shaped model layer (D42) remains prepared and unscheduled.

Headline inventory. 88 explicit word cases in `executeWord` at D54, seven token classes, four lookup layers, the named roots plus the generic and word roots, fourteen negative sentinels (`-1` anonymous sort, `-2` term, `-3` typed term, `-4` generic function, `-5` method, `-6` word, `-7` channel, `-8` event, `-9` channel state, `-10` step, `-11` predicate sort, `-12` class, `-13` guard, `-14` sort constructor), the evaluator slices, and the live per-process scheduler. The kernel is the fixed point: tensors, image append and lookup, text interning, `EntryIdentity`, root threading, stack operations, literals, the protocol dispatcher, and `run`.

The falsifiable test. For each stage: *a user adds X without editing F#.* If a new word family, sort constructor, method, guard, or control form still needs an F# case, the stage is not a protocol yet. The second test is that a protocol decision returns a verdict, never a silent false.

Order. D15 → W10 → a follow-on predicate and composite sort decision → D16 encoding → word metaobjects → sort metaobjects → generic functions and methods → the cell system → control refinement. The Metaobject Protocol is downstream of generic functions, not a shortcut around them. W10 (D19) landed the generic-function and method substrate; D21, D31, D35, D36, D39, and D41 landed the earlier stages; D43–D50 completed the cell-system layer; and D51 completed stage 5. Stages 6 and 7 landed as D50 (conditions and restarts) and D48/D49 (method combination and metaclasses).

Non-goals. A reflective tower (`eval` itself as a metaobject); a grammar protocol for the tokenizer and parser; the scheduler track; and exposing metaobjects to the child-facing sentence layer. D9′s domain binding remains that surface. Control refinement and the full cell system are no longer non-goals; they are landed and documented.

3.13 Done — Scale Basic English, stage 1 (W2.1, D18)

Status: applied. The sentence language is now a controlled grammar rather than an order-sensitive reading of whichever vocabulary categories appear. D18 refines D8: domain words still come from the image, but the sentence structure is closed and the executable projection is partial.

Grammar. `every`/`no` at the start; a TypeName subject; an optional `that` relative clause of conditions joined by `and`; a FunctionName verb; a required full stop. A boolean condition is a predicate phrase whose surface form begins with `is` (the tokenizer canonicalizes `is alive` to `isAlive`), or an explicit `is <field>`. A numeric condition is `has <field> <comparison> <integer>`.

Polarity. `every` projects to `Capability`; `no` projects to `Invariant`. The emitter renders the invariant as a negated postcondition under the same preconditions. This closes D8's deferred quantifier polarity for the two admitted quantifiers.

Staged refusals. D33 later made existential `some` executable: it projects to `Possibility` (a witness predicate and a satisfiability obligation). D24 completed numeric `has`: it is a typed condition end-to-end, not a staged refusal. `all` is refused with a direct instruction to use `every`. Articles, pronouns, `when`, `not`/`never`, and `or` have named refusals. `not`/`never` are replaced by sentence-initial `no`; `when` waits for event semantics; `or` waits for a precedence and disjunctive-obligation decision.

Pinned. Thirty-seven parser examples (valid corpus, grammar-valid projection refusals, fourteen named refusals, the invariant emitter path, and the inactive-word refusal) plus one Hedgehog property that generates valid sentences and checks their canonical pattern, subject, conditions, and verb. Suite: **275 checks (257 examples + 18 properties)** at W2.1. D24 later made numeric `has` executable and raised the suite to **400 checks**; the reference is `docs/prose-grammar.typ`; the decisions are D18 and D24.

Remaining, named. `or` after a precedence decision; `when`/`Transition` after event semantics. None of these is silently guessed in the meantime.

3.14 Done — W10, multiple dispatch (D19)

Status: done (applied as D19). `components/prelude/evaluator/dispatch.fs` owns the generic functions and methods, the type inference, the selection, and the two new words. `GenericFunctionsRoot` joins `SetsRoot` and `DeclarationsRoot` as execution state that a persistence reader re-threads.

Landed:

- `generic` and `method` words; a generic is `[GenericTag; name; arity; policy; method…]` (`-4`), a method is `[MethodTag; arity; body; specializer…]` (`-5`), both under the append-only D16 tag registry.
- Type-first selection: a typed operand uses its seal's declaration; an untyped operand infers the unique most-specific declared sort that contains it, and no unique minimum is `UnknownType`, never a guess.
- A static exact-specializer-tuple fast path under `MostSpecific`; the dynamic fallback is extension inclusion for a known sort and membership for `UnknownType`. Specificity is extension inclusion, and incomparable or extension-equal maximal methods are `AmbiguousDispatch`.
- Named diagnostics: `InvalidGenericArity`, `UnknownGeneric`, `GenericArityMismatch`, `MethodArityMismatch`, `NoApplicableMethod`, and `AmbiguousDispatch`.
- Twenty-two examples plus one property; suite 298 (279 examples + 19 properties).

3.15 Prepared — DSPy-shaped model-backed programs (optional, D42)

Status: prepared, optional, not scheduled, not implemented. This is a documentation-only track; the whole of it today is `docs/dspy.typ`, D42, a ledger row, and this section. No source, dependency, word, or test changed.

What it is. A native signature, one model-touching word, a pluggable adapter, and a host-side optimizer, in the shape of DSPy. The signature is a **projection of the `.scale` contract and the verification lattice**, not a second source of truth: the parsed sentence already names the subject, its typed fields, and the verb, and the lattice already describes the input space.

How it fits without a fundamental change.

1. **One model-touching word.** `predict` is an edge word; every model call goes through it and returns a named `Result` refusal, never an exception.
2. **The model is an effect.** The adapter is a host port passed at the run boundary, not image state. Calls are recorded in an optional replay cassette; the determinism law is stated over the recording or excludes the word by name.
3. **Fast search, slow gate.** The fast metric (type check, value-set membership, property tests) ranks candidates and never accepts. The slow metric is the existing `Smt.discharge` plus `Repair.accept`; `Unknown` and `Unavailable` never pass.
4. **Optimization appends.** The optimizer searches instruction text and examples only; it never edits the `.scale` document and never mutates the program. Its result is a new append-only artifact with provenance. This is fixed-spec mode (D11) applied to presentation.
5. **Constrained decoding is graded.** Only the structural fragment (field names, booleans, finite value sets) can drive a grammar. Predicate and constructor sorts are post-parse predicates and may answer `Undecided`; symbolic fields cannot be enumerated. No grammar is promised for a quotation.

Synergies. The parsed sentence is the signature; the lattice is the input schema; Hedgehog shrinking plus the repair bundle is a sharper reflection signal than a full trace; `Repair.accept` is the slow metric; the event-stream (D29) supplies record and replay; and the Metaobject Protocol (D16/D17) gives the signature a data representation without a new kernel case.

Contrasts. The verified theory stays the finite typed fragment; prompt text is outside SMT. Model calls are nondeterministic, so they need replay. Transcripts may contain user data and the image cannot delete, so they live in an optional redacted cassette, never in the image. The optimizer is host code, not a kernel primitive, and the core project takes no model SDK dependency. DSPy's `compile` is called **optimize** here; Scale's compiler is the emitter.

Promotion gate. A signature that projects to the document and the lattice; one `predict` word with named refusals and record/replay; a fast metric that is search-only and a slow metric that reuses the existing gate; an optimizer that appends a new artifact and never edits the specification; differential tests over a recorded transcript; and no change to a domain that does not opt in.

Non-goals. Training or fine-tuning; proving the model, the prompt, or the instruction; a reflective evaluator; network access in the kernel; provider-specific constrained decoding as a guarantee; unredacted transcripts by default; and the fast metric as an acceptance oracle.

3.16 Done — the full cell system (D43)

Memory direction. Static linear types are the likely memory mechanism (owner direction, 2026-10-28). D30′s dynamic ownership is the runtime precursor; this supersedes D30′s "static linear types" non-goal. The open design question is what becomes reclaimable in the append-only image, when, and whether checking is static or at the image boundary. No memory track is scheduled yet.

Status: A–G done. The encoding, closure, class/slot/instance, precedence/dispatch, method-combination, and metaclass stages landed as D44–D50. The design is `docs/cell-system.typ`; the glossary carries the short description of each term. Conditions and restarts (D50) landed.

The word and the surface. Use **class** for the cell-system concept. It is not the old `ClassTag` classification verdict: that is now `ValueTag`, and the word is `value-of`. A class's extension is a value set, and the code type for a value set remains `ValueSet`. **The surface is Scale Basic English, not tensor shapes.** The tensor entries in the design note are internal image encodings; a user never writes them. The exact grammar for class and slot declarations is a later decision; until it lands those sentences are not in the admitted corpus.

The design in one paragraph. A class is an image-resident metaobject that owns a name, a superclass list, a slot list, a metaclass, and a method-combination policy. A slot is an image-resident definition with a name, a value set, an optional default, and an initialization rule. An instance is a value with stable address identity and an appended state: the identity entry is written once, and every slot write appends a new state entry, exactly like channel state (D27). Class precedence is a deterministic linearization of the superclass graph; dispatch on a class is nominal, not extensional, so it is stable as instances are added. Methods gain a qualifier, and a method combination orders them. Closures are image-resident code cells with an image-resident environment chain, so lexical scope does not become host state. Metaclasses make classes instances of a class, with a finite cell-class closure terminating at a root metaclass.

Stages. Encoding and roots (D44), closures and lexical scope (D45), slots and instances (D46), class precedence and nominal dispatch (D47), method qualifiers and combination (D48), and metaclasses (D49) landed. Conditions and restarts (D50) landed. Each stage has an acceptance test in the design note. The full cell system is landed.

Boundary. The cell system uses runtime value sets, not the verification `TypeLattice.FieldType`; the two coordinate systems stay separate (D32). It adds no SMT obligation and no prompt/model dependency. The image stays append-only, execution state is never serialized, failures are values, and an undecidable slot type or dispatch is `Undecided`, never a silent false. The kernel grows only by the cell primitives; a new class, slot, method, combination, or metaclass is data.

Open decisions. Instances are identity plus appended state (D46); the growth policy for long-lived mutation remains a later decision. Class precedence is C3 (D47). The exact Basic English grammar for declarations and the level-model clarification that ordinary values may carry metaobject references as data are still open. Closures landed before method combination, as planned (D45).

3.17 Done — D44, cell-system encoding

Status: done (applied as D44). The encoding layer is data only: tags, tensor codecs, the class and instance-state roots, and persistence. No allocation, slot access, dispatch, method combination, or metaclass behavior landed.

What landed.

1. **Tags.** `-15` class definition, `-16` slot definition, `-17` instance identity, `-18` instance state, `-19` closure, `-20` environment frame, `-21` method combination, `-22` qualified method, `-23` metaclass.
2. **Codecs.** `components/prelude/evaluator/cell-system.fs` encodes and decodes each entry, with counts and safe optional identities; a malformed entry is `None`, not a crash.
3. **Roots.** `ClassTable` and `InstanceStateTable` in `EvaluationState`; both are association-vector pointers, re-threaded by a persistence reader.
4. **Persistence.** Every entry round-trips; the roots are execution state and are re-threaded, never serialized. No host metaobject store exists.
5. **Naming.** The method-combination policy is `Standard`, `Append`, `List`, or `LastResult`; `LastResult` is the Common Lisp `progn` combination named for what it does (D40).

Suite. 612 example checks and 53 granular tests.

Next. The behavior stages remain deferred: closures and lexical scope, slots and instances, class precedence and nominal dispatch, method combination, metaclasses, and conditions. The design note carries their acceptance tests.

3.18 Done — D45, closures and lexical scope

Status: done (applied as D45).

What landed. `let` creates a child frame; `lambda` captures the current frame in an image-resident closure; `apply` on a closure runs its body in the captured frame and restores the caller's frame. `lookup` reads the frame chain first, then the global binding table. `bind` and `define` stay global, and a bare quotation keeps dynamic scope. The current frame and saved caller frames are execution state; the internal return token restores the caller, and a return without a call is `UnmatchedReturn`. `QuotationRun` resets frames, a spawned actor starts at the global frame, and history rollback returns to the global frame (call-frame capture is a later change).

Suite. 623 example checks and 53 granular tests.

Next. The behavior stages remain deferred: slots and instances, class precedence and nominal dispatch, method combination, metaclasses, and conditions.

3.19 Done — D46, classes, slots, and instances

Status: done (applied as D46).

What landed. `class`, `slot`, `create-instance`, `slot-value`, and `set-slot-value` operate over image-resident class and slot definitions. An instance has a stable identity and an appended state; a write appends a new state and leaves the old one in the image. Effective slots are the class's own slots followed by its superclasses' slots, deduplicated by name; a cycle is skipped. The declarations are the lab surface (comma-separated names); the Basic English grammar is a later decision. Named refusals cover unknown class, unknown value set, unknown slot, missing slot, unbound slot, non-instance, non-class, and slot type mismatch.

Suite. 636 example checks and 53 granular tests.

Next. Class precedence and nominal dispatch landed as D47; then method combination, metaclasses, and conditions.

3.20 Done — D47, class precedence and nominal class dispatch

Status: done (applied as D47).

What landed. C3 precedence with `ClassCycle` and `InconsistentPrecedence`; `class` validates a new class before it enters the class root; effective slots follow the precedence list. `class-precedence`, `subclass-of`, and `instance-of` answer the class queries. The generic dispatcher accepts class specializers: an instance is a `ClassType`, a class specializer applies through the precedence list, specificity is precedence inclusion, the exact-tuple fast path uses class identity, and a class/value-set tie is `AmbiguousDispatch`.

Suite. 647 example checks and 53 granular tests.

Next. Method qualifiers and combination; then metaclasses and conditions.

3.21 Done — D48, method qualifiers and combination

Status: done (applied as D48). Methods carry Primary, Before, After, or Around, and the standard combination runs Around most-specific-first, all Before most-specific-first, the primary chain most-specific-first, and all After least-specific-first. `call-next-method` advances the segment; before and after methods have no next method; `next-method-p` answers a verdict. The chain and its continuations are execution state, never serialized. The suite pins the order, the around result, `NoNextMethod`, and the qualifier-sensitive replacement rule.

Suite. 653 example checks and 53 granular tests.

3.22 Done — D49, metaclasses and the cell-class closure

Status: done (applied as D49). `metaclass` declares an image-resident metaclass; `use-metaclass` re-declares a class with a named metaclass; `cell-class` answers instance to class, class to metaclass, metaclass to the root, and the root to itself. A user metaclass needs no F# case. The closure is finite and not a reflective tower. Stage G, conditions and restarts, remains deferred.

Suite. 660 example checks and 53 granular tests.

3.23 Done — D50, conditions and restarts

Status: done (applied as D50). A condition is an image-resident value with a kind, a name, and a message; a restart carries a name and a handler quotation. `with-handler` turns a failed body into an error condition and runs the handler; `signal` raises a condition, leaving it as a value when unhandled; `invoke-restart` runs a named restart handler. Handler frames are execution state, never serialized. `Repair.conditionWitness` reads the condition's name and message.

Suite. 668 example checks and 53 granular tests.

3.24 Done — D51, control refinement

Status: done (applied as D51). `quotation`, `if`, `apply`, and `define` are control words: image-resident `Word` metaobjects with either a default kernel primitive or a code cell as their implementation. `handler name control` installs a lexically scoped code cell; the default forms run in the caller's frame. A later declaration replaces a form without editing F#. The dispatcher checks control words before its built-in fallback tokens, and inactive control words raise `InactiveWord`. The kernel dispatcher now has 85 explicit word cases.

Suite. 688 example checks and 53 granular tests.

3.25 Done — D52, reference linearity and reclaim candidates

Status: done (applied as D52).

Goal. Start the owner-directed memory track by making the append-only image's reclaimable set visible without rewriting it.

What landed. `Reclaim.report` walks the conservative reference graph from explicit roots, `Store.Table`, `Store.VocabularyTable`, and the text cache. It classifies each entry as `Root`, `Unreferenced`, `Linear`, or `Shared` and returns the entries no live root can reach. `Reclaim.rootsOf` enumerates the execution-state roots, including frames, handlers, ownership values, and scheduler/channel roots; with more than one actor it originally treated every entry as a root, and D82 replaces that with every live actor's suspended state. The `reclaimable` word appends and pushes the candidate vector.

Boundary. A possible reference is any non-negative integral tensor element below `NextIndex`, so the report over-approximates the graph and never names a live entry. It can miss dead entries, and it never deletes one: the image stays append-only.

Suite. 688 example checks and 53 granular tests.

Non-goals. Physical deletion, a precise reference encoding, and static ownership types. Those are the remaining memory-track work.

3.26 Done — D53, opt-in compaction with stable addresses

Status: done (applied as D53).

Goal. Physically reclaim the entries D52 names without renumbering the image or rewriting a reference.

What landed. `compact` replaces the current in-memory image with a sparse successor that keeps only entries reachable from the live roots. Surviving addresses and the allocation frontier are unchanged. `compactForState` is the host entry point. Persistence now accepts a lexicon count below `NextIndex` and still refuses a count above it, duplicate addresses, out-of-range addresses, and text names that point at missing entries.

Boundary. Compaction is explicit and opt-in. It is never automatic; `reclaimable` reports, and `compact` acts. With more than one actor, every entry is a root, so compaction keeps the whole image.

Suite. 700 example checks and 53 granular tests.

Non-goals. Renumbering, automatic collection, precise reference tags, and static ownership types. Those are separate decisions.

3.27 Done — D54, static ownership contracts

Status: done (applied as D54).

Goal. Check a quotation's ownership discipline before it runs and report a named verdict rather than a silent pass.

What landed. `OwnershipTypes.checkQuotation` tracks one abstract input reference through a symbolic stack of `Unowned`, `Owned`, `Consumed`, and `NonReference` slots. It rejects a second consume, borrow after consume, use without ownership, aliasing through `duplicate`/`over`, and dropping an owned reference. It returns `Undecided` for words or variables it does not model. The `linearity` word pushes a tagged verdict tensor with a status and an interned reason.

Boundary. This is a conservative, local contract over the modelled ownership-and-shuffle subset, not a full effect system for the vocabulary. Undecided is never treated as linear.

Suite. 718 example checks and 60 granular tests.

Non-goals. Full type inference, effect signatures for every word, ownership transfer, borrow regions, and serializing ownership.

3.28 Done — D55, transition emission

Status: done (applied as D55).

Goal. Admit the event-triggered sentence that D18 deferred until event semantics existed, and give it its own executable pattern rather than reading it as a capability.

What landed. `Parser.parseTransition` admits `when <subject> <condition>, every <subject> <verb>.` The trigger's conditions are the precondition and the verb is the effect, so `SentencePattern.Transition` reaches the F# emitter as a guarded effect and the SMT obligation as a universal property. The trigger and effect must name the same subject (`TransitionSubjectMismatch`), the outcome must be universal (`TransitionOutcomeNotUniversal`), and the comma is structural (`ExpectedTransitionComma`). The tokenizer splits commas, the emitter names the property `when <subject> <verb>`, `generated/transition-sample.fs` is the pinned compiled artifact, and the repair corpus accepts `"Transition"` entries.

Boundary. A transition is a guarded effect over one subject, the same truth condition as `Capability` under a distinct pattern. A before/after state-change rendering needs a two-state domain model the specification and the verification lattice do not carry; it remains deferred.

Suite. 741 example checks and 60 granular tests.

Non-goals. State-change emission, temporal logic, effect signatures, anaphora, and any new kernel case or dependency.

3.29 Done — D56, witness-guided guards for large domains

Status: done (applied as D56).

Goal. Let the refiner narrow a domain whose lattice has no finite coordinates, using the solver's counterexample rather than an exhaustive universe.

What landed. `TypeLattice.isFinite` splits the finite and symbolic regimes. `TypeLattice.candidateGuardsFromWitness` seeds the weakest guards that exclude the unrefined counterexample; `Refiner.refine` widens each numeric seed outward and bisects to the weakest proved threshold, or records `Coverage = 0` because coverage is not measurable on a symbolic domain. A missing witness and a missing field are named refusals, and no proved guard yields `Ok None`.

Boundary. The proof is still `pre ∧ guard ∧ ¬post` by Z3, never sampling. The search is bounded at `2^30`; multi-field and non-linear guards are out of scope. The finite path and its pinned artifacts do not move.

Suite. 753 example checks and 60 granular tests.

Non-goals. Multi-field or conjunctive guards, non-linear arithmetic, exhaustive symbolic coverage, and temporal or relational guards.

3.30 Done — D57, violation reporting phase P0

Status: done (applied as D57).

Goal. Make every grammar refusal one blamable value: which layer detected it, which rule it broke, which constituent is at fault, what was expected and found, and whether the fault is the input's or the fixed specification's.

What landed. `violations.fs` defines `Layer`, `Blame`, `Span`, `Witness`, and `Violation`; `ofParseError` maps every `Parser.ParseError` case to a named rule, layer, and blame; and `Violations.parse` returns `Result<Sentence, Violation>`. `Blame` separates `Input` (Scale Basic English forbids it) from `CoverageGap` (the fixed specification does not cover it), and a few refusals carry a direct repair hint.

Boundary. P0 wires only the parser. `Span` is the offending canonical token, not a character offset, because the tokenizer keeps no source offsets. `Witness` declares its cases and produces none; the sort, unification, guard, SMT, and discourse detectors, unsat cores, and the minimum-edit repair search are phases P1–P5.

Suite. 791 example checks and 60 granular tests.

Non-goals. The later violation phases, any schema change, any new dependency, and any change to an accepted sentence or a pinned artifact.

3.31 Done — D58, disjunction

Status: done (applied as D58).

Goal. Admit `or` with a decided precedence and a disjunctive obligation, completing the sentence layer's `and`/`or`/`no`/`some`/`when` logic.

What landed. `and` binds tighter than `or`; the relative clause (and a transition trigger) is a disjunction of conjunctions. The executable shape is one `Condition.Disjunction` atom over atomic conjunctions, flattened to its atoms for a single clause, so every existing sentence and pinned artifact is byte-identical. The F# precondition is `||` over parenthesized && clauses, the SMT obligation is `MkOr` over `MkAnd`, the repair gate evaluates the DNF, and the canonical `(a and b) or (c)` text round-trips through `parseConditionText` without changing the corpus schema.

Boundary. `or` joins conditions, not sentences; a bare field is still `ExpectedCondition`, and a transition's outcome stays universal. Nested parentheses beyond the canonical DNF text, `not`, and implication are out of scope.

Suite. 801 example checks and 60 granular tests.

Non-goals. Nested disjunction text, distributing a disjunction across a transition's effect, and a corpus-schema or pinned-artifact change.

3.32 Done — D59, higher-order unification

Status: done (applied as D59).

Goal. Extend the unification pillar from first-order terms to the Miller-pattern fragment of higher-order unification, with lambda terms, beta-reduction, and an explicit undecided verdict.

What landed. `LambdaTag = -15` adds a binder; application reuses the compound encoding. A flex term (a variable applied to a distinct-atom spine) binds the variable to the curried abstraction of the rigid side, with the existing occurs-check. Lambda terms alpha-rename a parameter; substitution is capture-avoiding by refusing a capture. `unify-verdict` pushes `[UnificationVerdictTag; status; reasonId]` (0 unified, 1 not, 2 undecided); `unify` keeps its 1/0 result and reports `UnificationUndecided` for a non-pattern case. `lambda-term` and `reduce` build and beta-normalize terms.

Boundary. This is the Miller-pattern fragment plus the first-order fallback, not full Huet unification: no imitation or projection search, no non-pattern spine, no dependent types, and no eta-conversion. `Undecided` is the named boundary, never a false.

Suite. 814 example checks and 60 granular tests.

Non-goals. Full higher-order unification, dependent types, eta/delta conversion, and any change to the first-order surface or pinned artifacts.

3.33 Done — D60, tensor algebra beyond scalars (W11)

Status: done (applied as D60).

Goal. Make arithmetic and comparison elementwise over tensors of equal shape, and stop reading a non-scalar through its first element.

What landed. `binaryOperation` requires equal shapes, sequences the per-element results, and returns a tensor of that shape; a scalar is the rank-0 case, so the old scalar behavior is unchanged. A shape mismatch is a named `ShapeMismatch`, never a silent broadcast or first-element read. `divide` propagates `DivisionByZero` from any element. `ScalarRead.readScalar` refuses a non-scalar. Vector addition, matrix elementwise multiplication, the zero-divisor refusal, both shape-mismatch directions, the scalar case, and the non-scalar read are pinned by tests.

Boundary. This is elementwise algebra, not matrix multiplication or contraction, and there is no scalar broadcasting. The tensor shape/data invariant is unchanged.

Suite. 823 example checks and 60 granular tests.

Non-goals. Broadcasting, matrix products, tensor contraction, and any change to the shape invariant or the pinned artifacts.

3.34 Done — D61, violation reporting phase P1 (semantic stages)

Status: done (applied as D61).

Goal. Give the semantic stages the same blamable diagnostic as the parser, so a sortal, binding, agreement, dispatch, guard, or concurrency failure says which layer, rule, constituent, and party is at fault.

What landed. `Violations.ofEvaluationError` maps every `EvaluationError` case; `Violations.ofGuardVerdict` maps a holding, failed, or undecided guard. Stack/shape/term errors are `Structural`, unknown words `Lexical`, ownership `Binding`, sorts/dispatch/classes/guards `Sortal`, undecidable unification `Agreement`, and channels/events/conditions `Semantic`. A bad program is `Input`; an `Undecided` membership, dispatch, unification, guard, or budget is `CoverageGap`. The value carries the rule (the error case name), the offending token or entry, and expected/given where the constructor carries them.

Boundary. P1 reports values the pipeline already produces; it does not change an accepted program or the error type. P2–P5 (SMT unsat cores/models, discourse rollback, minimum-edit repair, presentation) remain open.

Suite. 853 example checks and 60 granular tests.

Non-goals. The later violation phases, a schema change, a new dependency, and any change to an accepted program.

3.35 Done — D62, violation reporting phase P2 (SMT models and unsat cores)

Status: done (applied as D62).

Goal. Attach the solver's evidence to an SMT diagnostic: a counterexample model when the obligation is falsified, and the labelled unsat core when it is proved or a possibility is refuted.

What landed. `Smt.Discharge` carries the verdict and the core; `Smt.dischargeWithCore` takes its verdict from the shared plain `solve` and re-checks only an unsat answer with `AssertAndTrack` to read `UnsatCore`; `Smt.discharge` remains that plain solve, so the production path pays for no second query. The tracked names map back to `condition <n>`, `refinement <guard>`, `lattice guard <guard>`, and `postcondition`/`negated postcondition`. `Violations.Witness` now carries `Model` or `UnsatCore`, and `Violations.ofSmtDischarge`/`ofSmtOutcome` map the verdict to a `Violation`: `Falsified` and `NoWitness` are `Input`, `Unknown` and `Unavailable` are `CoverageGap`, and `Proved`/`Witnessed` are not violations. The encoded formula, the satisfiable model selection, the `.smt2` bytes, and every accepted program are unchanged; a tracking query that does not reproduce the unsat answer leaves the core empty rather than inventing a verdict.

Boundary. P2 reports evidence the SMT stage already produces; it does not add detectors, change the acceptance gate, or turn `Unknown` into a pass. P3–P5 (discourse rollback, minimum-edit repair, presentation) remain open.

Suite. 898 example checks and 60 granular tests.

Non-goals. The later violation phases, a schema change, a new dependency, and any change to an accepted program.

3.36 Done — D63, supervision and token deadlines (W5)

Status: done (applied as D63).

Goal. Give the cooperative scheduler the preemption that D22 deferred: a token deadline per actor incarnation, and a supervisor that decides what a failed or exhausted child becomes.

What landed. `Consumed` counts executed tokens and is not reset by a fairness yield; `Deadline` is the optional lifetime bound. A supervised child that reaches it exits with `TimeoutExit`. Three strategies (`one-for-one`, `one-for-all`, `rest-for-one`) and `supervise` install the policy and spawn children; `Permanent`/`Transient`/`Temporary` decide whether a child restarts, a restart is a fresh identity from `ActorSpec.Quotation`, and a bounded restart window (`MaxRestarts` within `WithinSteps`) ends the supervisor's subtree with `SupervisionLimitExceeded`. An unsupervised `spawn` still aborts the run on error, and a blocked actor consumes nothing, so a deadline never fires on it. **Boundary.** The supervisor's own failure surfaces as the run diagnostic; it is not propagated to a grandparent supervisor.

Non-goals. Wall-clock deadlines, priorities, mailbox limits, dynamic child specifications, true parallelism, distribution, and actor migration.

Suite. 930 example checks and 73 granular tests.

3.37 Done — D64, actor handles are private capabilities

Status: done (applied as D64).

Goal. Make the actor address private: a program must not be able to target an actor by guessing a number.

What landed. `spawn`/`supervise` push an image-resident handle (`ActorHandleTag = -28.0`), not the child's address. `kill`/`resume` resolve the handle through `SchedulerState.ActorHandles` and refuse a number, a forged entry, a foreign entry, or a retired handle with `NotAnActorHandle`. `actors` reports one status per actor and no addresses. The mapping is execution state, never serialized; a restart takes a fresh identity and a fresh handle. (D65 later adds `self`, so a self handle exists; `kill` still refuses self.)

Boundary. A handle grants `kill`/`resume` to its holder, like a PID; it adds no authentication or revocation. Channel addresses remain numeric store entries and are not covered.

Non-goals. Cryptographic unguessability, cross-run handles, delegated or revocable handles, capability transfer beyond the ordinary stack value, and channel-address privacy.

3.38 Done — D65, actor mailboxes, self, and links

Status: done (applied as D65).

Goal. Complete the actor model: a private mailbox per actor, message passing by handle, a self handle, and links that deliver abnormal exits.

What landed. `self` pushes the actor's handle. `send-to` queues in the target's mailbox (a hidden channel); `receive-message` dequeues or blocks like `receive`. `link`/`unlink` maintain a mutual link. An abnormal exit delivers a pending exit to every live linked actor, consumed before its next token; a normal exit signals nobody. Links are per incarnation and are cleared on termination. The mailbox id never reaches the stack; mailboxes reuse the W6/D27 channel state and are unbounded.

Boundary. No monitors, selective receive, or named registry at D65; D66 adds trap-exit, D67 adds monitors and the registry, and D68 adds selective receive. A mailbox id can appear in a manual-resume error, but it is never a stack operand.

Non-goals. Monitors (D67), selective receive (D68), a named process registry (D67), bounded mailboxes, distribution, and priorities.

3.39 Done — D66, trapped exits

Status: done (applied as D66).

Goal. Let an actor observe a linked abnormal exit instead of dying with it.

What landed. `trap-exit` sets the current actor's flag. A linked abnormal exit becomes an `[ExitSignalTag; reasonCode]` mailbox message; `exit-signal?` and `exit-reason` inspect it. The signal uses the same delivery/queue path as `send`, so a blocked receiver wakes with it. A non-trapping actor keeps the D65 pending-exit kill, and a normal exit signals nobody.

Boundary. Only links trap; an explicit `kill` is not a link signal and still terminates. Selective receive remained a non-goal at D66; D67 adds monitors and the registry, and D68 adds selective receive.

Non-goals. Selective receive (D68 adds it), bounded mailboxes, distribution, and priorities.

Status: done (applied as D67).

Goal. Let an actor observe another actor's exit one way, and give actors stable names without exposing addresses.

What landed. `monitor` returns a per-run reference; any exit of the target delivers a `[DownTag; ref; reasonCode]` message to the watcher once, and the monitor is consumed. `demonitor` removes it; a forged or foreign reference is `NotAMonitor`. `down?`, `down-ref`, and `down-reason` inspect the message. `register`/`unregister` map a text name to an actor; `whereis` returns its handle; a live name is exclusive and a terminated actor's names are removed. Both maps are execution state.

Boundary. A monitor fires on the target's exit, not on a restart as a new identity. An explicit `kill` fires the target's monitors; D68 makes a cascading kill fire the monitors of every terminated descendant. Selective receive was the one actor-model operation still absent at D67; D68 adds it.

Non-goals. Selective receive (D68 adds it), bounded mailboxes, distribution, and priorities.

3.41 Done — D68, closing the actor boundaries

Status: done (applied as D68).

Goal. Finish the actor model: selective receive, a supervisor's own supervisor as the next decision point, monitors on cascading kills, and a single send target for channels and mailboxes.

What landed. `receive-matching` scans a mailbox with a predicate quotation, returns the oldest match, and leaves the rest queued in order; with no match the actor blocks, and a later send wakes it to rescan. `send` and `send-event` accept an actor handle and route it to that actor's mailbox; `receive` and `close` stay channel-only. A supervisor that spends its restart budget stops its children with `ShutdownExit` and exits with the limit error to its own supervisor, so the root sees the error only if the chain reaches it. A cascading kill fires the monitors of the target and every terminated descendant. `SelectiveWaiters` is execution state and is never serialized.

Boundary. One predicate quotation, no timeout, no multi-clause receive. `receive-matching` is refused inside a `quotationRun`. Bounded mailboxes, distribution, and priorities remain non-goals; with D68 the actor model is otherwise complete.

Suite. 940 example checks and 73 granular tests.

3.42 Done — D69, one actor-word guard

Status: done (applied as D69).

Goal. Make the quotation-run and set-literal guards one policy, cover the actor words added by D65–D68, and close the nested-quotation bypass.

What landed. `EvaluatorQuotationRun.actorWords` is the shared list; the set literal reads it. Both pre-scan the whole body, so `{ ... }` and `apply` cannot smuggle `receive`, `receive-message`, `receive-matching`, `send`, `send-to`, `close`, `link`, `unlink`, `trap-exit`, `monitor`, `demonitor`, `register`, or `unregister` past the guard. The state-boundary test now also resets `SelectiveWaiters`, and a killed selective waiter is removed from the map.

Boundary. A quotation that merely names an actor word is refused even if it never applies it. Read-only actor words (`self`, `whereis`, `down?`, `exit-signal?`) stay available; event constructors stay available until `sync` commits.

Suite. 953 example checks and 74 granular tests.

3.43 Done — D70, discourse rollback (P3)

Status: done (applied as D70).

Goal. Make a failed utterance a discourse event: publish its violation and roll back on an input fault.

What landed. `Discourse.utter` runs one attempt and returns `Accepted`, the resulting state, and the violations. An `Input` violation at `Structural` or above rejects the attempt and returns the pre-utterance state; a `CoverageGap` is published without blocking. `Discourse.sameObservable` defines the restored state (stack, program, counters); the image stays append-only. The Hedgehog property `discourse rollback restores the state after an input fault` pins it.

Boundary. This is a host policy over one completed attempt, not yet an actor/ChannelEvent publication. P4 (minimum-edit repair search) and P5 (presentation) remain open; so do character offsets.

Suite. 962 example checks and 75 granular tests.

3.44 Done — D71, minimum-edit repair search (P4)

Status: done (applied as D71).

Goal. Turn a failed sentence into the nearest legal Basic English sentence under the fixed grammar, and rank candidates for a sortal check.

What landed. `RepairSearch.search` explores one edit at a time (delete, insert, replace, adjacent swap) over the canonical tokens and stops at the first distance with a parsed candidate. `repair` returns the best candidate; an accepted sentence is its own zero-edit repair. `searchWith` adds a layer check and a distance bound, so a sortal caller can rank several candidates. Punctuation `./,` is in the edit alphabet even though it is not a vocabulary entry.

Boundary. Default distance is one; larger repairs need an explicit bound. Sortal ranking uses the caller's check, not a semantic search. P5 (presentation) remains open.

Suite. 976 example checks and 75 granular tests.

3.45 Done — D72, presentation layer (P5)

Status: done (applied as D72).

Goal. Turn a `Violation` into an explanation a non-logician can act on.

What landed. `Presentation.render` gives a layer-aware headline, the token span, expected/given, the blamed party, the witness (model, unsat core, or shrunk sentence), and the repair. `renderTechnical` adds the stable rule id; the ordinary rendering leaves it out. `withNearest` fills a missing repair from the P4 search.

Boundary. The span is the canonical token; D76 adds optional source character offsets through `Violations.parseText`. The rendering is presentation only and changes no verdict. P0–P5 are complete.

Suite. 986 example checks and 76 granular tests.

TODO: The showcase and the CLI `--diagnose` mode render a parser violation, a nearest repair and a discourse rejection; D74 adds the table surface, sink and `--table`; D76 adds character offsets. Wire the SMT/discourse detectors.

3.46 Done — D73, decision-table promotion

Status: done (applied as D73).

Goal. Promote the prepared D13 table into an emitted per-row property surface without reopening the Basic English grammar.

What landed. `DecisionTable.TableBinding` supplies one body per outcome; `DecisionTable.obligationFor`/`obligations` expose each row as an `Smt.Obligation`; `DecisionTableEmitter.emit` renders one module per row with the row's guards as the precondition and `rowN` as the body. `DecisionTable.explain` maps a `NoMatch` or `Ambiguous` selection to one `Violation`; it is an adapter, not a sink. `DecisionTable.check` still proves completeness and (for `Unique`) disjointness. The two-row demo is pinned in `snapshots/decision-table.verified.txt`.

Boundary. Host-level surface only: no new word, parser, or vocabulary change. The artifact assumes the host supplies `rowN`. D74 adds the Basic English table surface and the sink.

Suite. 1018 example checks and 76 granular tests.

3.47 Done — D74, the Basic English table surface and sink

Status: done (applied as D74).

Goal. Give the promoted table a Basic English document surface and a common diagnostic sink.

What landed. A table document is `table <name>`, an optional `policy:` line, transition rows, and `end`. The row guards come from the trigger conditions, the outcome from the verb, and the subject from the first row; input sorts are inferred. `DecisionTableSurface.parse/toTable/binding` produce the host table without changing the one-sentence `.scale` document. `Diagnostics.Sink` collects `Violations` and `addSelection` adapts `DecisionTable.explain`; `render` and `renderTechnical` use `Presentation`. The showcase parses and selects a table, then renders the sink.

Boundary. A document format, not a new sentence grammar. Disjunctive rows and `NamedGuard` rows are refused; the host supplies the outcome body. `--table <path>` and `--diagnose <sentence>` are the CLI modes; character offsets remain TODO.

Suite. 1035 example checks and 76 granular tests.

3.48 Done — D75, reserved actor system messages

Status: done (applied as D75).

Goal. Make the actor model OTP-compatible enough for an Elixir-to-Scale compiler by reserving one runtime-handled message class.

What landed. `SystemMessageTag` marks `[operation; payload…]`; the runtime checks it in `deliverMessage` before any user delivery. The five operations are Suspend, Resume, Code-Change, Status, and Shutdown. `Actor.Suspended` is execution state; a suspended actor leaves the ready queue, keeps its state, and receives user messages only into its mailbox queue. Resume requeues a ready actor and wakes a blocked actor only when its mailbox has a queued message. Code-Change requires a suspended actor, reads a quotation, replaces the program, clears the pending error, and leaves the actor ready but still suspended. Status writes `[SystemStatusTag; statusCode; suspended; mailboxDepth]` to an optional reply channel. Shutdown captures the target subtree's links, terminates it with `ShutdownExit`, propagates the exit to live linked actors, and fires the monitors of every terminated actor. Self-targeted control operations are refused.

Boundary. Only actor mailboxes accept the class; an ordinary channel refuses it. The mailbox structure, MOP, persistence, and linear regions are unchanged. No new user word exists. Status query is runtime data, not a user callback.

Superseded direction (D151). The Elixir-to-Scale compiler is no longer the target. An Elixir implementation is a separate instance connected by the actor IPC; the system-message class remains useful for instance control.

Suite. 1044 example checks and 76 granular tests.

3.49 Done — D76, character offsets for source violations

Status: done (applied as D76).

Goal. Put the offending source range in a violation when the caller has the original sentence.

What landed. `Violation.Span` now carries optional `Start` and `End` character offsets, with an exclusive end. `Violations.parseText` tokenizes and parses the source sentence and fills the range from the offending surface form; `parse` remains the token-only API and leaves the offsets absent. `surfaceForm` resolves a canonical token through the active vocabulary, and `Presentation` renders `characters start-end`. The showcase and `--diagnose` use the source-text entry point.

Boundary. A repeated surface form uses its first occurrence because the tokenizer does not retain a token index. A multiword canonical token uses its full vocabulary form. Semantic violations without a source sentence keep offset-free spans. Offsets are presentation data, never a verdict.

Suite. 1051 example checks and 76 granular tests.

3.50 Done — D77, a syntactic discourse contradiction detector

Status: done (applied as D77).

Goal. Produce a `Layer.Discourse` violation from a direct contradiction between two committed statements.

What landed. `Discourse.contradicts` detects Every/No and Some/No over the same subject, ordered conditions, and verb. `Discourse.contradiction` finds the first committed statement a candidate contradicts and returns a `Contradiction` violation blaming the input. The showcase parses a committed statement and a candidate and renders the result. Transitions are excluded.

Boundary. The detector is syntactic: no SMT, no paraphrase matching, no condition reordering, and no transition contradictions. It is a named production detector for the Discourse layer, not a complete consistency checker.

Suite. 1052 example checks and 76 granular tests.

3.51 Done — D78, the SMT violation in the showcase sink

Status: done (applied as D78).

Goal. Put an SMT counterexample through the same diagnostic sink as the parser, table, and discourse examples.

What landed. The showcase creates the demo encoding from `DemoDomain.lattice` and `DemoDomain.canJumpExpression`, discharges an empty-condition `Capability`, converts the result with `Violations.ofSmtDischarge`, and adds it with `Diagnostics.addOption`. The encoding is disposed with `use`. The sink renders the semantic violation in the showcase.

Boundary. One falsified property from the demo domain; no CLI SMT mode, no new encoding, no change to the acceptance gate. `Unknown` and `Unavailable` remain coverage-gap violations through the same adapter.

Suite. 1051 example checks and 76 granular tests.

4. Recommended sequence

This is the dependency-ordered sequence. The track list that used to sit here is history: the completed items are recorded in the decision records and in sections 1–3 above. A subsystem starts only when its prerequisites are frozen, because the later work encodes the earlier representation.

4.1 Subsystems and their prerequisites

ID	Subsystem	Prerequisites	State
F1	Image, identity, store, tensors	none	frozen foundation
F2	Evaluator, stack, control	F1	complete (D1, D4, W5)
F3	Set-theoretic sorts and value sets	F1	complete (D12, D14)
F4	Unification and terms	F3	complete (D15, D59)
F5	Scheduler, actors, channels	F1, F2	complete (D22, W5–W9, D63–D69, D75)

ID	Subsystem	Prerequisites	State
F6	Event-stream, history, rollback	F2	complete (D29)
F7	Ownership and memory	F1, F2, F5	**frozen (D80–D90)**
F8	Metaobject protocol and cell system	F1, F2, F3, F4	complete (D16/D17, D21, D31, D35/D36, D39, D41, D43–D51)
F9	Parser and Basic English	F1, F3	complete (D8, D18, D55, D58, D69)
F10	Emitter, verification, diagnostics	F3, F4, F9	complete (D5–D11, D23, D25, D55–D62, D70–D78)
F11	Persistence	F1, F7	complete through D87/D88
F12	Native .NET port	F2, F5, F6, F7, F8, F9, F10	**P1 next (unblocked by D90)**
F13	Self-hosting and bootstrap	F9, F10, F12	not started
F14	AOT and compiler boundary	F12 phase 2	rule fixed (D84); implementation pending
F15	Optional: BEAM parity, distribution, DSPy, tracing GC	F5, F7	not scheduled

4.2 The chain

```
F1 -> F2 -> F5 -> F6 -> F7 -> F12 -> F13
F1 -> F3 -> F4 -> F8
F3 -> F9 -> F10 -> F13
F7 -> F11
F12 -> F14
```

The graph shows the critical path, not every edge: F12 also consumes F8 and F10, and F13 also consumes F9 and F10.

4.3 Remaining work, in order

1. **M1 — freeze the memory representation (complete, D90).** The image and `Tensor` representation are frozen, the backing is managed, values are shared immutably with promotion, copy-on-send is rejected, and the evaluator is not routed through the arena. D80–D90 carry the tier.
2. **P0 — record the port start.** `docs/native-dotnet-port.typ` is the plan. Add the decision record and ledger row for P1 when the first IR code lands.
3. **P1 — executable IR and VM (current).** Lower the kernel subset to the lowered DU, run it on a small VM, and differentially test it against the token interpreter. No assembly emission. Acceptance is the port document's.
4. **P2 — the SRM writer.** `System.Reflection.Metadata` / `System.Reflection.Emit`, one method per code object, interpreter fallback for live redefinition. It is a JIT-only build tool, excluded from the AOT publish; the native entry returns a named refusal (D84). The writer contains no language `match`.
5. **P3 — self-typing and self-hosting.** Only after P2; separately decided. This is where the dynamic effect seams are solved, not in P1.
6. **Optional.** BEAM operational parity (copy-on-send and a per-process tracing GC), DSPy, and an unmanaged arena backend. None is on the critical path.

7. **Distribution (D133).** A target track, planned: node identity, remote send/receive, and partition/failure semantics. The `scale_transport_send` ABI seam exists.

4.4 What may run in parallel

- Documentation, tooling, and Vestige. They touch no core representation.
- The D84 AOT boundary can be prepared with P1: interface, stub, and build wiring. It is not wired to the writer until P2.
- Port planning and the differential-test corpus inventory may start during M1. Any port code that names `Tensor`, `Store`, stack layout, history, or dispatch waits for M1.
- Do not parallelize M1 and P1 code, and do not start F13 before P2.

4.5 Current position

Tracks A–J are complete through D90. F7/M1 is frozen: image/Tensor representation, managed backing, shared immutable values with promotion, no copy-on-send, and no evaluator arena routing. The arena contract is a host seam, not a second representation. The next item is P1. The suite counts are the header line of this document.

Actor IPC direction (D151). The unified Julia/Elixir language (D93) is dropped. Scale has no shared source language for Julia and Elixir; each implementation is a separate instance in its host language, and instances integrate through the actor model's IPC. The core language remains Scale's own concatenative surface; Basic English and the prose surface remain its front ends. The new track is documented in `docs/actor-ipc.typ`; it does not change P1, the image contract, or the suite.

Wire protocol direction (D152). Scale implements its own actor wire protocol, Cap'n Proto-inspired and in-repo, with C++, Rust, and Go as the three major implementation references and Protobuf and OCapN as secondary references. The first scope is the envelope and portable payloads; a schema compiler, RPC pipelining, three-party handoff, and distributed GC wait for a consumer. D153 narrows the compatibility position: the frozen subset is compatibility-gated, not a blanket claim. The plan is `docs/binary-format.typ`; it does not change P1, the image contract, or the suite.

Schema tooling direction (D153). The actor message keeps one frame with a logical envelope/body split. The wire follows Cap'n Proto for the frozen subset, with compatibility gated by a reference conformance test. The schema layer takes Protobuf's descriptor set, plugin contract, field numbers, and reserved ranges as build-time metadata mapped to ordinals; `oneof` maps to a union. Unknown fields are opt-in; `Any` is a typed union plus a registry. This refines D152 without changing P1, the image contract, or the suite.

First actor IPC slice (D154). The `ipc` component implements a single-segment Cap'n Proto subset and a Unix-socket actor host. A listener receives a numeric envelope, delivers it to the root actor's mailbox, runs the actor, and returns the reply; `--wire-listen` and `--wire-connect` expose it. One connection, one message, root actor only. Capabilities, discovery, remote supervision, far pointers, and reference conformance remain. The suite is 1355 example checks, 33 granular cases, and 91 discovered tests.

Worker capability and remote supervision (D155). The envelope gains a send/reply/monitor/monitor-ack/down operation. A capability names a non-root worker; the root monitors it; the client receives the reply and `down normal` when the worker terminates. A missing socket is the named `Unreachable` failure. The scheduler's `continueAfter` now preserves the threaded store when no ready actor remains, so the Down message is not lost; the S2 integration test is its regression test. Discovery, authentication, far pointers, and reference conformance remain.

BEAM wire harness (D156). `adapters/elixir` implements the actor wire subset, a supervised peer, and a BEAM listener with a real worker process. Both cross-language directions pass and eight

ExUnit tests pass. This is not a language port. The old architecture remains; a full BEAM port and a Rustler NIF over the C ABI are deferred until the Prelude is in Scale and a native kernel artifact can be tested.

Language-level remote send (D157). `send-to` recognizes a host-registered remote handle `[RemoteHandleTag; instance; capability]` and records a `TransportRequest` in `SchedulerState.PendingTransport`; it performs no I/O. The host drains the queue, reads the message entry, and transports it.

Remote reply delivery (D158). `LanguageTransport.deliverPending` sends the request, adds the numeric reply to the store, delivers it to the sender's mailbox, and resumes the sender. `send-to; receive-message` returns 42; the host also sees `down normal`. One sender per drain; numeric replies only.

Wire golden vectors and reference interop (D159). `components/ipc/vectors/` pins six positive and two negative vectors with a Cap'n Proto schema. `capnp encode` generates the reference bytes; the F# test decodes them and re-encodes byte-for-byte; `check.sh` re-encodes every text vector and decodes the Elixir encoder's bytes. The frozen subset is conformance-tested; far pointers and multi-segment messages remain out.

Typed term IR (D160). The live program becomes `Term list`; a quotation is `Quote of Term list`; `apply` splices. The kernel owns the core term encoding, the term-list loop, the structural and stack-effect verifier, the `VerifiedTermProgram` receipt, and the verify entry. The Prelude owns surface elaboration, word signatures, language type rules, effects, dynamic-code policy, macros, and lowering. The domain SMT verification stays separate. The migration P0–P8 is in `docs/term-ir.typ`.

Term shape and portable document (D161). P1 lands `TermIr` and `TermPortable` in the kernel: the core term variants, `TermProgram`, canonical JSON, and six positive plus two negative vectors. The vectors round-trip byte-for-byte. No term executes and no stack is typed; P2 adds the verifier and receipt.

Kernel verifier and receipt (D162). P2 lands `TermVerification`: it re-checks the canonical term document, finite numbers, non-negative slots/captures/arities, non-blank word and closure names, and every quotation body at its path. `VerifiedTermProgram` is opaque and carries a SHA-256 digest of the canonical program. `scale_verify` is export nine and ABI version 2; it writes the canonical `term-receipt` JSON. Stack typing, stack underflow, and `apply` remain P4/P3.

Token-to-term boundary (D163). P3a adds `EvaluatorProgram`: finite numeric tokens become `LiteralNumber`, other tokens become `Word`, a non-finite number is refused, and `ofText` reuses the token boundary. `toTokens` is the temporary projection for the kernel bridge. `EvaluationState.Program` is still `string list`; P3b changes the active state, P3c adds code values, and P3d removes the projection and text execution. P3b's contract is recorded (D164); the migration is next.

Typed control state (D165). P3b is implemented. `EvaluationState.Program` and the caller frames on `HandlerFrame`, `MethodFrame`, and `CombinationFrame` are `Term list`; the loop executes `LiteralNumber` and `Word`, and any other active term is an invariant failure. `EvaluatorProgram.toKernelTokens` is the temporary total projection for those two shapes, and `ofTokensOrFail` is the total construction form. The control words, the token-vector history, and the kernel bridge project at their boundary and convert back with `ofTokens`, so text semantics are unchanged. The differential corpus keeps `Case.Program` as `string list` and converts only at `runInterpreter`; `KernelState.Program` and `LoweredAbstraction.Program` remain `string list`.

P3c adds tagged quotation values, P3d removes the projection and token-vector history, and P4 adds stack typing.

Quotation code values (D166). P3c is implemented. A quotation is `[QuoteTag; documentId]`, where the document is the canonical `term-program` JSON for the body. `EvaluatorCode.encode` creates it; `EvaluatorCode.decode` returns its terms. `{ ... }` collects nested `Quote` terms, `Quote` execution pushes a code value, and `if`, `apply`, `define`, `control`, `lambda`, closures, actor bodies, handlers, methods, predicates, guards, and set quotations decode code values. History stores code values in its program and current-term fields. The legacy text reader remains only as a P3c migration seam; P3d removes it and the remaining token projections.

Text side channels removed (D167). P3d removes the legacy text quotation reader, `toTokens`, and the token-vector history. A text tensor is not code; `EvaluatorCode.decode` accepts only `[QuoteTag; documentId]`, and history stores code values in both its current-term and program fields. Surface clause grammars and the kernel bridge keep their token projections as parser/bridge boundaries.

Pipeline IR (D168). The prose surface parses `value |> stage` left-associatively. `Pipeline` records `LastArgument` (the default) or `FirstArgument`; F# is the compatibility case for the former and Elixir for the latter. Lowering desugars to `Call` with the value appended or prepended, and the portable expression document preserves the node and style. A stage may carry whitespace arguments (`x |> f a`); general partial application and currying outside a pipeline remain open.

Explicit partial application (D170). A call argument may be `_`; `subtract(_ 10)` becomes a one-hole closure. Supplied arguments are evaluated once at creation, holes bind when applied, and a call with no holes remains a normal `Call`. This is explicit partial application, not automatic currying. The portable expression document preserves `Partial`, and lowering creates a lambda with hole parameters and captured supplied values. `apply` handles the resulting kernel closure.

Reverse pipe (D171). `f <| x` is supported as `ReversePipeline` for F# package compatibility, with `f a <| x` appending the value and right associativity for chains. It is discouraged in new Scale code; the forward pipeline or a direct call is preferred. The portable document preserves the node, and lowering appends the value to the left call.

Phase 0 refactoring, step 1 (D94). `EvaluationError` is now a 41-case union with typed fault families; both `describeError` and `Violations.ofEvaluationError` reproduce the previous messages and rule strings. The 250 updated call sites compile. The snapshot baselines were stale, not the code: `33cf99d` disables Verify's shared-prefix rule and corrects the decision-table artifact to the integer rendering. SageFs reports 174 passed and 0 failed.

Phase 0 refactoring, step 2 (D95) — historical record; see D98–D105 for current state. The three-layer boundary is recorded and `components/kernel/scale_abi.h` plus `components/kernel/Scale.Kernel.fsproj` defined ABI version 1 with a then-unimplemented body; D99–D105 now implement image execution, primitive dispatch, and image transfer. The kernel is the abstract machine; dispatch, classes, unification, set algebra, ownership, guards, conditions, and supervision policy are Prelude code. The typed D94 errors stay inside the kernel and are encoded as `(code, message)` only at the ABI edge. The first type split is landed: `kernel/types.fs` is the core half in the kernel assembly and `image-types.fs` is the image, word, and condition half in the Scale assembly; `bases/scale-cli/scale.fsproj` references the kernel project. The kernel assembly also compiles `epoch.fs`, `tensor.fs`, `index-transformation.fs`, `lexicon.fs`, `store.fs`, `scalar-read.fs`, `stack.fs`, `components/prelude/evaluator/common.fs`, `components/prelude/evaluator/stack.fs`, `components/prelude/evaluator/history.fs`, and `components/prelude/evaluator/quotation-run.fs`. At that point `scale_run` forwarded to the host adapter through `host_dispatch`; D101 later

moved execution into the kernel. The halves are exposed as `Types.Core` and `Types.Image`, and the suite reports 174 passed. The remaining split is the prelude and edge halves.

Pattern matching, step 3 (D96). The runtime pattern IR and the ground-subject matcher are landed in `components/prelude/evaluator/pattern.fs`, with the `subject { pattern -> body | ... } match` surface and the fresh, pin, repeated, and shadowing roles. The IR is `pattern-ast.fs`; the portable form is `pattern-portable.fs`, a closed versioned JSON document with canonical constructor kinds and pinned named predicates, an open subject is matched through the unifier, and a `when` guard runs a named predicate over the bound values. The best-effort reachability, redundancy, and closed-kind exhaustiveness analysis is `PatternAnalysis`; pattern templates are a separate open form, refused by the closed reader, and `PatternElaboration` turns a template plus hole arguments into a closed pattern. The general expression AST and its canonical JSON v1 document are landed, and the package document carries units, exports, imports, capabilities, and pinned registries. The suite reports 1138 checks.

General IR design (D97). `docs/general-ir.typ` records one portable IR for expressions, patterns, packages, kinds, predicates, sorts, conditions, dispatch, and actor messages. Pattern matching is its first typed sub-language. Phase 1 (pattern IR and portable pattern document) is landed; Phase 2 (general expression and quotation nodes) is next.

P1 first slice. `kernel/lowered-ir.fs` defines the small opcode set (literals, local slots, late-bound word calls, apply, named failure) and `kernel/lowered-abstraction.fs` runs the literal/slot/call subset against the same dispatcher as the token interpreter. The differential, slot, cache-guard, and bare-quotation and closure `Apply`, both lowered `if` branches, and `with-fields` are in the suite. The closed nominal node universe remains.

Boundary correction (D98). After the audit, the portable pattern/ expression/package IR, pattern analysis, history, quotation-run, and the common and stack evaluator slices moved out of the kernel into the Prelude assembly. The lowered abstraction no longer reads ownership state; cache-guard validity is a callback. `runtime-types.fs` now carries actor/channel/scheduler/ownership/ supervision/monitor/method types, `EvaluationState`, and the full `EvaluationError`; `epoch.fs`, the lowered IR/VM, and channel store encoding also moved to the Prelude. The kernel now holds the core types, tensor/store primitives, the lowered IR/VM, and the ABI, with a boundary fitness check. The suite reports 1251 checks. `scale_init` now validates and retains the prelude image in the kernel, then delivers it to the host adapter via `host_dispatch` (`scale/prelude`) during the transition; `scale_run` resolves an exported unit and executes it with the kernel abstraction (D101).

Total expression lowering (D119). `EvaluatorLowering.lowerExpression` is now total over the closed constructor universe: every `Expression` node either lowers or is declined with a named `LoweringRefusal`, so the phase-2 emitter can fall back to the interpreter by category instead of discovering an unsupported node as a run-time `Fail`. A `call` over a word target lowers and is differentially tested against the token interpreter. `let` writes both a frame binding and a fresh slot (D122): a bare word resolves lexically in both engines, while `lookup` still reads the frame, and nested lets shadow. `lambda` now lowers to a named unit with slot-bound parameters and a kernel-recorded arity (D123); every enclosing lexical name is captured conservatively, and `CallClosure` refuses an underflow before the body runs. A shared differential corpus (D124) now runs eighteen token programs through both engines and compares results and failure families. `pattern-match` now compiles to inspection, branches, and slots for ground subjects (D125): literals, wildcards, fresh variables, constructors, and typed patterns lower; pin, repeated, shadowing, and guards remain named refusals. The declaration nodes and non-word call targets remain declined.

Port review (D127). The D119–D126 slice is recorded with its process deviations and open debts in `docs/technical-debt-register.typ`. `EnableWriter` is now Debug-only by default, so a Release publish cannot accidentally include the JIT-only writer; a Release JIT run that needs the writer tests passes `-p:EnableWriter=true`.

P2 started (D126, D128). The IL writer is a JIT-only backend behind `EnableWriter`. The harness (D128) makes the emitted method take and return the real `KernelState<EvaluationState>`, call the AOT-safe `WriterRuntime`, and share `LoweredAbstraction.step` for value instructions, stack access, dispatch, and failure. The emitted assembly is saved, loaded, and invoked in the granular suite; an unsupported opcode refuses to the interpreter. Units as methods and the remaining opcodes are the next slice.

Language specification (D129). `docs/specification.typ` is now the normative entry point for language questions. It covers the core and the extensions and records the status of every feature; the roadmap, ledger, and decision records remain the plan and the history.

Open-question decisions (D130–D132). The alpha reload is SageFs `rebuild=true`. The writer's next slice is units as methods. Pattern roles, guards, and declaration nodes stay interpreter-only. The first adapter, C, landed with its conformance harness (D136). The surface stays word-oriented: no comments, one-word text literals, whitespace as the separator. `set-subset` is the value-level subset. Basic English and Z3 verification wait. Sequences and recursive types are target; multisets and co-products wait for a consumer.

Polylith workspace (D137). The repository is a Polylith workspace: `workspace.edn` lists the nodes, the `.fsproj` graph is the dependency truth, and `just poly-check` enforces the boundaries, cycles, and interfaces. Stage 1 moved `components/kernel` and `components/adapter-c`; stage 2a moved the app to `bases/scale-cli` and added `projects/scale` as the publish target. Stage 2b splits the Prelude, verification, demo, and development nodes. Nu joins as its own node after the conversion, wrapped by `nu-host`; both sides are NativeAOT, so the C ABI is the seam. D181 records the host and Gaia-on-ImSim plan; no Nu base, project, or build lane is integrated yet. D182 records the colocated jj workspaces: `Scale-code` is the sparse Scale-only checkout, and the main workspace keeps Nu. D138 adds the module dependency map; D139 lands the full split: `core`, `typing`, `prelude`, `verification`, `demo`, and `cli` are components, `development/scale` holds the suite, and the base is `entry.fs`. D183 adds `shell-host`, the process effect boundary for the prose shell; D184 adds `book-host`, the book/spell loader and prose `cast` adapter.

Demo content root (D141). `Demo.demoDirectory` resolves `components/demo/demo` in a checkout (the `workspace.edn` marker) and `<base>/demo` in a publish payload. `specificationPath`, `runLoop`, and the suite baselines use it, so the copied payload runs the showcase; two checks pin the mapping.

Prose surface (D142–D145). `components/core/prose-parser.fs` implements the sentence-shaped program surface: lowercase `match`/`when`/ `otherwise`/`close`, full stops, no `end`, and `.prose` files. The comma delimits the multiple-choice construct, so `match e, when p then e, otherwise e.` is one prose sentence; the line form with one clause per line is the alternative. Names are lowercase words separated by spaces and canonicalized through the vocabulary (`is alive` -> `isAlive`); a free multiword name is an `of` phrase (`state of player`), and kebab-case is refused. Text literals may contain spaces. A nested match is closed by `close`. Both forms parse to `ExpressionAst.PatternMatch` with named `ProseError` refusals; thirty-three checks pin the forms, names, text, nested matches, the sentence sequence, and the refusals. A one-sentence program keeps its expression and two or more become a `Block`; the sequence lowers with `EvaluatorLowering` and runs on the kernel `LoweredAbstraction`, with the final stack top as the result.

First test batch (D148). P1 adds a Hedgehog property comparing the interpreter and the lowered abstraction over generated arithmetic; P2 adds a property comparing the inline and line prose forms over generated numeric matches; S1 pins the lowered differential corpus as a Verify snapshot. The example suite is 1355 (1313 examples, 42 properties) and the granular suite is 33 cases (11 snapshots); discovery is 91 tests. P3 and the remaining batch wait.

Prose file reader (D149). `scale --prose <path>` reads a `.prose` file, parses it with the demonstration vocabulary, lowers it, and runs it on the kernel `LoweredAbstraction`; the final stack top is printed and a refusal returns a non-zero exit code. The CLI mode is `Cli.ProseFile`; the runner is `Entry.runProseFile`, which delegates to `ProseShell.runFile`. The `run` and `capture` effects are Stage 1a host words (D183); books and spells are Stage A (D184): a `.book` compiles to a package, `main` makes a spell, `scale --book` and `scale --cast` run or call it, and prose `cast` calls a book directly. The `with` connective is implemented (D187); a REPL remains the later D147 work.

5. Gotchas

Serena F# LSP unavailable or the project is not loaded. Report it immediately; do not silently fall back to `read`. Run `just lsp-check`; if it fails, run `just lsp-restart` (it restarts this user's `fsautocomplete` processes, never SageFs), then retry. Serena is the only LSP navigation path. Its first workspace load can take a minute; slow is not hung.

- **`dotnet` PATH.** `dotnet` is on the normal PATH: bash (login and interactive) and fish prepend `/usr/share/dotnet` and `~/.dotnet/tools`, so `dotnet` and `fsautocomplete` resolve. A shell started before that change can still carry the old environment; open a new shell.
- **F# LSP.** The client needs an `lsp.fsharp` block pointing at `~/.dotnet/tools/fsautocomplete` (0.85.4) with `DOTNET_ROOT=/usr/share/dotnet` and `initialization.AutomaticWorkspaceInit: true`; `just lsp-check` verifies the executable, the configuration, and a real document-symbol request. The server itself is verified: `initialize` returns `FsAutoComplete 0.85.4`. Without `AutomaticWorkspaceInit`, requests fail with `Couldn't find <file> in LoadedProjects`. Its first workspace load runs MSBuild and can take a minute; slow is not hung.
- **Serena and the LSP.** Serena is the only LSP navigation path. `just lsp-check` runs `serena project health-check`; `just lsp-restart` restarts this user's `fsautocomplete` processes, never SageFs. Symbol overviews and definitions are reliable. Reference queries have returned loose and false matches on `fsautocomplete` 0.85.4, so confirm a reference with a definition lookup or by reading the resolved region; a returned list is a hint, not a complete call graph. **The LSP has no read operation**: locate with Serena, then `read` the region.
- **SageFs reports `Degraded` on `bases/scale-cli/scale.fsproj`.** The session can say "runtimeconfig.json names no framework version" while `bin/Debug/net10.0/scale.runtimeconfig.json` correctly names `Microsoft.NETCore.App` 10.0.0. That is a SageFs false positive; a rebuild and `hard_reset_fsi_session` may not clear it. Evals and tests still run, so proceed and report the tool's message; never restart a shared daemon for it.
- `bases/scale-cli/scale.fsproj` lists `Compile` items explicitly; **new files must be added there, in dependency order** (`types.fs` first, `bases/scale-cli/entry.fs` last).
- **ValueSet roots are execution state (D4).** `SetsRoot`, `DeclarationsRoot`, and `GenericFunctionsRoot` are pointers into image-resident association vectors; persistence serializes the image data only, so a reader must re-thread all three roots. Lookup is latest-write-wins; a declared domain only grows. `[ … ]` is an extension literal (the body is evaluated), `S :Name seal` is postfix, `:Name` is a reference, and the `[ ?x : … ]` comprehension surface is not implemented (D14; the filtering word landed as D34).

- **Generated `.fs` artifacts are checked in and compiled** (`generated/`, `components/demo/demo/generated/`). They are the emitter's pinned output: a run that re-emits must be byte-identical to what is checked in, or the repository goes dirty. The emitter's **module name is a parameter** — two emitted modules must have flat, distinct names (F# will not accept one nested under another's name).
- **`==>` binds tighter than `&&`.** A multi-clause emitted precondition must be parenthesized (`(a && b) ==> c`); the emitter does this, and leaves a single clause bare so the pinned golden output is unchanged.
- **Z3.** The NuGet package is `Microsoft.Z3` **4.12.2** (the package version, not the CLI version); `/usr/bin/z3` is 4.16.0 and serves as the offline fallback. `Status` is `SATISFIABLE`/`UNSATISFIABLE`/`UNKNOWN`; a `Model` is structured data (`Evaluate`), not text. `Microsoft.Z3.Expr.ToString()` prints SMT-LIB — use it for the `.smt2` artifact so text and discharged AST are one source. Dispose `Context`/`Solver` (`use`). `Status.UNKNOWN` must be reported, never swallowed as a pass.
- **W15 persistence.** `System.Text.Json` serializes F# record properties, but a `private` F# record type serializes as `{}` — keep the bundle/corpus DTOs public. The corpus writer emits values as strings (`{ name, value }`), while the checked-in corpus uses native JSON booleans/numbers; `readCorpus` accepts both. The gate counts only `Expected = true` entries in the regression check (the target proof is what makes the original failure pass), and it rejects `Unknown`/`Unavailable` rather than treating either as an acceptance.
- **A git repository exists** (created with the initial commit). Commit each verified state; do not refactor on an uncommitted tree.
- Sandbox has no network by default; adding the Hedgehog NuGet package may require approval — another reason Q2 matters early. **(Resolved: Hedgehog 1.0.0 added.)** API gotchas: F# API is under `Hedgehog.FSharp` (qualify types as `Hedgehog.Gen<'a>` / `Hedgehog.Property<'a>`); the `property` CE needs a `let!` before `return` or it degrades to `Property<unit>`; `Property.reportBool` gives a `Report` with `Status` = OK/Failed/GaveUp; exceptions inside property bodies are swallowed into `Failed` — wrap with try/with to surface the cause; `List.zip` throws on unequal lengths.
- `Tensor.append` appends at the end; stack top = last element (`swap` was once a silent no-op because of this — it is fixed and tested).
- Ownership of vocabulary truth: `Vocabulary.gameVocabulary` is the *seed* only; all consumers must derive via `VocabularyInImage.readVocabulary` and respect `IsActive`.

6. Done — repair-model redesign (Option D)

Status: done (applied as D79). The full analysis is `docs/repair-model-analysis.typ`; this section records the implemented model.

Target model. Typed proposals are separate from advice. An implementation proposal cannot carry a specification edit; a specification proposal requires an exploratory capability; a sentence suggestion is advice, not a proposal. The fixed-spec gate consumes a frozen specification and an implementation proposal.

What landed.

- `repair.fs` → `fixed-specification-acceptance.fs`, module `FixedSpecificationAcceptance`.
- `repair-search.fs` → `nearest-sentence.fs`, module `NearestSentence`.
- `ImplementationProposal` replaces `Patch`; there is no `SpecEdit` field.
- `FrozenSpecification` carries a private `SpecificationIdentity`.

- `SpecificationProposal` is constructed only from a private `SpecChangeCapability`; `specChangeCapability FixedSpecification` refuses.
- `accept` takes the frozen specification and an implementation proposal, checks their identities, then runs the sorted-body, proof, corpus, and differential rules.
- `NearestSentence.nearest`/`candidates`/`nearestWith` replace the old search names; `NearestSentence.advice` produces `Violations.Advice`.
- `Violation.Suggestion` replaces the string `Violation.Repair`; `Presentation.withNearestSentence` fills advice and renders `Suggestion:`.
- `AcceptanceBundle` replaces `RepairBundle`; the bundle JSON wire names are unchanged.

Boundary. The capability is a private-constructor value, not a type-theoretic linear type; fixed mode has no capability. The acceptance gate never calls sentence search. There is no combined verdict-plus-advice facade; `--diagnose` remains sentence advice and `--fixed-spec` remains implementation acceptance. Historical records keep their old names.

Suite. 1251 example checks and 17 granular tests.
